

PRIVACY-PRESERVING NEURAL NETWORKS IN FEDERATED LEARNING

A PROJECT REPORT SUBMITTED IN PARTIAL FULFILLMENT OF THE

REQUIREMENTS FOR THE AWARD OF THE DEGREE OF

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING (AI & ML)



By

Batch - 10

S. Deepika(21JG1A4253)

J. Sravanthi(21JG1A4219)

D.Manasa Praneetha(21JG1A4209)

N.Keerthana(21JG1A4237)

Under the esteemed guidance of

Mr. P. Shiva

Assistant Professor

CSE(AI & ML) Department

Department of Computer Science and Engineering (AI & ML)

GAYATRI VIDYA PARISHAD COLLEGE OF ENGINEERING FOR WOMEN

[Approved by AICTE NEW DELHI, Affiliated to JNTUK Kakinada]

[Accredited by National Board of Accreditation (NBA) for B. Tech CSE, ECE & IT-Valid from 2019-22 and 2022-25]

[Accredited by National Assessment and Accreditation Council (NAAC) Valid from 2022-27]

Kommadi, Madhurwada, Visakhapatnam-530048

2021-2025

GAYATRI VIDYA PARISHAD COLLEGE OF ENGINEERING FOR WOMEN
Department of Computer Science and Engineering (AI & ML)



CERTIFICATE

This is to certify that the project report titled Privacy-Preserving Neural Networks in Federated Learning is a bonafide work of following IV B.Tech -II students in the Department of Computer Science and Engineering (Artificial Intelligence and Machine Learning), Gayatri Vidya Parishad College of Engineering for Women affiliated to JNT University, Kakinada during the academic year 2024-25, in partial fulfillment of the requirement for the award of the degree of Bachelor of Technology of this university.

S. Deepika(21JG1A4253)

J.Sravanthi(21JG1A4219)

D.Manasa Praneetha(21JG1A4209)

N. Keerthana(21JG1A4237)

Internal Guide

Mr.P.Siva

Asst.Professor

Dept.of CSE - AI & ML

Head of the Department

Dr.Dwiti Krishna Bebarta

Professor & HOD

Dept.of CSE - AI & ML

External Examiner

ACKNOWLEDGEMENT

The satisfaction that accompanies the successful completion of any task would be incomplete without the mention of people who made it possible and whose constant guidance and encouragement crown all the efforts with success.

We feel elated to extend our sincere gratitude to **Mr.P.Siva** , Assistant Professor for encouragement all the way during analysis of the project. His annotations, insinuations and criticisms are the key behind the successful completion of the thesis and for providing us all the required facilities.

We would like to take this opportunity to extend our gratitude to Project Coordinator, **Dr. K. Purushotam Naidu**, Associate Professor of Computer Science and Engineering(Artificial Intelligence & Machine Learning) for making us follow a defined path and for advising us to get better improvements in the project.

We express our deep sense of gratitude and thanks to **Dr. Dwiti Krishna Bebarta** , Professor and Head of the Department of Computer Science and Engineering (Artificial Intelligence & Machine Learning) for his guidance and for expressing his valuable and grateful opinions in the project for its development and for providing lab sessions and extra hours to complete the project.

We would like to take this opportunity to express our profound sense of gratitude to **Dr. R. K. Goswami**, Principal and **Dr. G.Sudheer**, Vice Principal for allowing us to utilize the college resources thereby facilitating the successful completion of our thesis

We are also thankful to both teaching and non-teaching faculty of the Department of Computer Science and Engineering (AI&ML) for giving valuable suggestions from our project.

S. Deepika(21JG1A4253)

J.Sravanthi(21JG1A4219)

D.Manasa Praneetha(21JG1A4209)

N. Keerthana(21JG1A4237)

TABLE OF CONTENTS

Topics	Page No
Abstract	i
List of Figures	ii
List of Tables	iii
List of Screens	iv-v
Symbols & Abbreviations	vi
1. Introduction	1-4
1.1 Motivation	1
1.2 Problem definition	2
1.3 Objective of Project	3
1.4 Limitations of Project	3
1.5 Organization of Documentation	4
2. LITERATURE SURVEY	5-17
2.1 Introduction	5
2.2 Existing System	13
2.3 Disadvantages of Existing system	14
2.4 Proposed System	15
2.5 Conclusion	17
3. REQUIREMENT Analysis	19-32
3.1 Introduction	19
3.2 Software Requirement Specification	18
3.2.1 User requirement	20
3.2.2 Software requirement	21
3.2.3 Hardware requirement	22
3.3 Content diagram of Project	23
3.4 Algorithms and Flowcharts	24
3.5 Conclusion	32
4. DESIGN	33-44
4.1 Introduction	33
4.2 DFD / ER / UML diagram (any other project diagrams)	34
4.3 Module design and organization	42
4.4 Conclusion	44
5. IMPLEMENTATION & RESULTS	45-85
5.1 Introduction	45
5.2 Explanation of Key functions	45
5.3 Method of Implementation	49

5.3.1 Forms	50
5.3.2 Output Screens	82
5.3.3 Result Analysis	83
5.4 Conclusion	85
6. TESTING & VALIDATION	86-99
6.1 Introduction	86
6.2 Design of test cases and scenarios	86
6.3 Validation	93
6.4 Conclusion	96
7. CONCLUSION:	97-98
First Paragraph - Project Conclusion	97
Second Paragraph - Future enhancement	97
8. REFERENCES	99-100

ABSTRACT

Federated Learning is one of the most promising privacy-preserving distributed model training methods, mainly for sensitive domains like the healthcare domain. Federated Learning, however, brings about an immense trade-off between model performance and strong privacy guarantees by embedding DP methods into it. Whereas Federated Learning also brings a decentralized learning architecture that does not involve raw data sharing, its mechanisms like DP inject noise in gradients to bind the information leakage while negatively impacting utility in the models. This work examines how the number of clients, DP mechanisms, and several data characteristics influence FL's performance using Scatter-CNN and RDP(Renyie Differential Privacy). Balancing privacy and performance is difficult when the scenarios include small or non-i.i.d. Datasets leading to more profound performance degradations. In this work we used some standard datasets like MNIST and FMNIST and by changing the parameters we monitored the performance of the model and balanced the model performance and the privacy.

Keywords: Federated Learning, Differential Privacy, RDP, Gaussian Noise, Scatter-CNN

LIST OF FIGURES

S.no.	Figure No	Figure Name	Page No.
1	Fig 3.1	Content Diagram	23
2	Fig 3.2	Data Extraction	27
3	Fig 3.3	Data Preprocessing	28
4	Fig 3.4	Federated Learning System	29
5	Fig 3.5	Differential Privacy Flowchart	30
6	Fig 3.6	Scattering Transform Flowchart	31
7	Fig 4.1	Use Case Diagram	34
8	Fig 4.2	Class Diagram	35
9	Fig 4.3	Sequence Diagram	38
10	Fig 4.4	Activity Diagram	42
11	Fig 4.5	Module Design	44
12	Fig 5.1	MNIST Dataset Class Distribution	83
13	Fig 5.2	FMNIST Dataset Class Distribution	84
14	Fig 5.3	MNIST Accuracies of Clients 1,5,10	84
15	Fig 5.4	FMNIST Accuracies of Clients 1,5,1	85

LIST OF TABLES

S.no	Table No.	Table Name	Page No
1	Table.1	Literature Survey	10-12
2	Table.2	Software Requirements	21
3	Table.3	Hardware Requirements	22
4	Table.4	Comparison B/W MNIST AND FMNIST	47

LIST OF SCREENS

S.no	Screen Name	Screen No.	Page No.
1	Screen 5.1	Selecting Dataset, No of Clients and Image file	
2	Screen 5.2	Prediction 1 (MNIST)	
3	Screen 5.3	Prediction 2 (FMNIST)	
4	Screen 6.1	MNIST Client-1	
5	Screen 6.2	MNIST Client-1 Prediction	
6	Screen 6.3	MNIST Client-5	
7	Screen 6.4	MNIST Client-5 Prediction	
8	Screen 6.5	MNIST Client-10	
9	Screen 6.6	MNIST Client-10 Prediction	
10	Screen 6.7	FMNIST Client-1	
11	Screen 6.8	FMNIST Client-1 Prediction	
12	Screen 6.9	FMNIST Client-5	
13	Screen 6.10	FMNIST Client-5 Prediction	

14	Screen 6.11	FMNIST Client-10
15	Screen 6.12	FMNIST Client-5 Prediction
16	Screen 6.13	Dataset Validation
17	Screen 6.14	Number of Clients Validation
18	Screen 6.15	Image Validation

SYMBOLS AND ABBREVIATIONS

F	Federated Learning
DP	Differential Privacy
RDP	Renyi Differential Privacy
Output_size	Number of output classes
K	Total number of clients in federated learning
H	Height of input images
W	Width of input images
lr	Learning rate for model training
E	Number of local training epochs per client
C	Fraction of clients participating per round
ϵ	Differential privacy (DP) budget
δ	Privacy loss probability
q	Subsampling rate

1. INTRODUCTION

In the era of big data and artificial intelligence, the demand for machine learning models that can process vast amounts of data while preserving user privacy has increased significantly. Traditional machine learning techniques rely on centralized data collection, where data from multiple sources is aggregated in a single location for model training. However, this approach raises serious concerns regarding data security, privacy breaches, and compliance with regulatory frameworks such as GDPR. Federated Learning (FL) has emerged as an alternative paradigm that enables decentralized model training while ensuring that data remains with its respective owners.

Despite its advantages, existing FL models face critical challenges such as privacy leakage due to gradient-based attacks, inefficiencies in communication due to high bandwidth consumption, and the inability to personalize models for diverse users. Additionally, Differential Privacy (DP), a commonly used privacy-preserving technique, introduces trade-offs between accuracy and security, often degrading model performance when privacy constraints are strictly enforced.

1.1 MOTIVATION

In the digital age, data privacy has become a significant concern, particularly in fields where sensitive user data is involved, such as healthcare, finance, and personalized services. Traditional machine learning models rely on centralized data collection, which raises privacy risks and regulatory compliance challenges. Federated Learning (FL) has emerged as an innovative approach that allows multiple institutions or users to collaboratively train a model while keeping data localized, thereby reducing privacy risks. However, existing FL frameworks still suffer from potential data leakage, inefficiencies in communication, and a lack of personalized learning mechanisms. As organizations continue to adopt FL, concerns over adversarial attacks, model security, and heterogeneity in local datasets create new challenges. Addressing these challenges requires a framework that balances privacy, computational efficiency, and model

performance, ensuring that FL can be effectively implemented in real-world applications. This project is driven by the need to develop a Privacy-Preserving Federated Learning Framework that not only enhances security and efficiency but also provides customization options to improve model accuracy for different users. By leveraging Convolutional Neural Networks (CNNs) and fine-tuning FL parameters, this framework aims to achieve higher accuracy, reduced communication overhead, and enhanced data security while maintaining a personalized approach to model training.

1.2 PROBLEM DEFINITION

Federated Learning presents a decentralized model training approach where multiple participants contribute to training a global model without exposing their private data. While this method enhances security, it introduces several challenges. Federated Learning (FL) has emerged as a powerful paradigm for decentralized model training, allowing multiple participants to collaboratively improve a global model while preserving data privacy. However, despite its advantages, FL still faces several challenges that impact its effectiveness and security. One of the most significant concerns is privacy leakage. This loophole raises concerns about data confidentiality, especially in domains dealing with sensitive information such as healthcare, finance, and personalized services.

To overcome these limitations, this project proposes a Privacy-Preserving Federated Learning Framework that enhances security, optimizes efficiency, and improves model adaptability for different user groups. By using Convolutional Neural Networks (CNN) for image classification tasks and fine-tuning FL parameters, this framework aims to achieve higher accuracy, reduce communication overhead, and enhance data security while still providing flexibility for user-specific adaptations. This project is particularly significant in applications where privacy is paramount, such as healthcare, finance, and smart devices. By introducing a robust FL framework that maintains privacy without compromising accuracy and efficiency, this project contributes to the advancement of secure, scalable, and adaptive machine learning solution.

1.3 OBJECTIVE OF THE PROJECT

The primary objective of the project is to develop a Privacy-Preserving Federated Learning Framework that enables secure and efficient decentralized model training across multiple participants while addressing key challenges in privacy, communication, and personalization. To achieve this, the framework will incorporate DP techniques to safeguard against adversarial attacks and minimize the risk of data leakage during training. Optimizing model update exchanges will enhance communication efficiency, reducing bandwidth consumption and improving overall system performance. Another crucial aspect of this framework is local model personalization, which will allow participants to fine-tune the global model based on their unique data distributions, ensuring better adaptability and accuracy. The effectiveness of the proposed framework will be rigorously evaluated through extensive experimentation and comparative analysis with existing federated learning techniques to demonstrate its superiority in terms of privacy preservation, efficiency, and model personalization.

1.4 LIMITATIONS OF THE PROJECT

While the proposed framework aims to overcome several FL limitations, certain constraints remain:

- **Computational Overhead:** The integration of privacy-preserving mechanisms, such as differential privacy and secure model aggregation, increases computational complexity. Devices with limited processing power may struggle to handle local model training efficiently.
- **Privacy-Accuracy Trade-off:** Strong privacy measures, such as differential privacy, introduce noise into the training process to protect user data. This can lead to a slight reduction in model accuracy, especially when dealing with small datasets. Balancing privacy and performance remains a challenge
- **Scalability Concerns:** As the number of participating clients increases, efficiently managing model updates, optimizing network traffic, and maintaining consistent model performance become increasingly difficult. Larger deployments may require additional optimizations to sustain efficiency.

- **Security Risks from Malicious Clients:** Although the framework prevents raw data exposure, adversarial attacks such as model poisoning or Byzantine failures could compromise training integrity. Additional mechanisms may be required to filter out unreliable client updates.
- **Energy Consumption:** Running federated learning algorithms on edge devices (e.g., smartphones, IoT devices) may consume significant energy, affecting battery life and long-term usability.
- **Integration Complexity:** Deploying the framework in real-world applications, such as healthcare or finance, requires seamless integration with existing IT infrastructures. Compliance with data regulations (such as **GDPR**) may introduce additional challenges.

1.5 ORGANISATION OF THE DOCUMENTATION

To ensure a comprehensive understanding of the project, the documentation is structured as follows:

- Chapter 1: Introduction, outlining the motivation, problem statement, and objectives.
- Chapter 2: Literature survey, reviewing existing federated learning frameworks and their limitations.
- Chapter 3: Requirement analysis, detailing software and hardware requirements, as well as system design specifications.
- Chapter 4: Design methodology, including architecture diagrams and model customization strategies.
- Chapter 5: Implementation and experimental results, demonstrating the efficiency of the proposed framework.
- Chapter 6: Testing and validation, including performance evaluations and security assessments.
- Chapter 7: Conclusion and future enhancements, summarizing findings and discussing potential improvements.

2. LITERATURE SURVEY

2.1 INTRODUCTION

This literature survey reviews existing FL approaches, highlighting their strengths and limitations in terms of privacy protection, computational efficiency, and adaptability to heterogeneous client data.

In their study titled “Personalized Privacy-Preserving Framework for Cross-Silo Federated Learning,” Van-Tuan Tran et al.[1] The PPPFL framework combines differential privacy (DP) with federated learning (FL) and leverages GANs to generate synthetic data with privacy guarantees. Meta-learning is employed for personalized model adaptation, addressing data heterogeneity. PPPFL is compared with several FL algorithms, including FedAvg, FedNova, FedProx, SCAFFOLD, FedMeta, SDA-FL, AvgPush, ProxyFL, and Dis-PFL. Experimental results show PPPFL outperforms these baselines on multiple datasets, achieving accuracies of 96.40% on MNIST, 91.99% on F-MNIST, 80.00% on CIFAR-10, and 48.00% on CIFAR-100.

In their study titled “Federated Learning with Differential Privacy,” Adrien Banse et al.[2] Using MNIST, FMNIST, and a medical dataset, this paper investigates how federated learning (FL) with differential privacy (DP) affects model performance. For MNIST and FMNIST, CNN models are utilized, whereas medical data is subjected to logistic regression. DP is implemented using FedAvg and gradient perturbation with Opacus. The findings indicate that adding DP lowers accuracy and slows convergence, particularly on small and non-i.i.d. datasets. The accuracy of MNIST decreases from 95% (without DP) to 75% (with DP). With reduced privacy budgets, the medical dataset exhibits considerable variation and performance degradation.

In their study titled “Towards Adaptive Privacy Protection for Interpretable Federated Learning,” Zhe Li et al.[3] By using differential privacy (DP) in conjunction with selective noise injection, the AGP method for Federated Learning (FL) strikes a compromise between interpretability and privacy protection. It maintains good accuracy while reducing privacy leaks by using the ResNet18 model with SGD

optimization. The usefulness of AGP is demonstrated by experiments conducted on the MNIST, Blood, and Animals datasets in both IID and non-IID scenarios. Accuracy levels are 94.01% (MNIST), 81.59% (Blood), and 70.21% (Animals) in IID settings and 85.22% (MNIST), 68.44% (Blood), and 57.13% (Animals) in non-IID settings. With a better accuracy-privacy trade-off, AGP performs better than conventional DP techniques.

In their study titled “Personalized Federated Learning With Differential Privacy and Convergence Guarantee,” Kang Wei et al.[4] In order to improve privacy and convergence, the RDP-PFL system integrates Rényi Differential Privacy (RDP) with Personalized Federated Learning (PFL) using meta-learning. Convergence bounds for both convex and non-convex loss functions are derived, and RDP-SGD is used for training. MLP, CNN, and ResNet-18 model experiments on MNIST and CIFAR-10 provide excellent performance with respective accuracies of 96.40% and 80.00%. The system dynamically modifies the privacy budget and model size to strike a balance between privacy and learning performance. Effective privacy preservation and model adaptation are confirmed by the results, which validate the theoretical findings.

In their study titled “Design and implementation of privacy-preserving federated learning algorithm for consumer IoT,” Bin Zhao et al.[5] To improve cooperation and data security, the suggested FL system for smart home equipment combines blockchain technology with differential privacy. Using the MNIST dataset, a CNN model with normalization layers and ϵ -differential privacy noise is created for digit classification. Using convolutional and fully connected layers, the model has a batch size of 64, a learning rate of 0.01 and a privacy parameter ϵ that ranges from 1 to 10. According on experimental results, the suggested normalizing method performs better than conventional batch normalization, obtaining greater accuracy while preserving robust privacy protection.

In their study titled “Privacy preserving federated learning based on partial low-quality data,” Huiyong Wang et al.[6] The PPFL-LQDP framework enhances privacy and performance in federated learning by using the distributed Paillier cryptographic mechanism for secure aggregation while allowing partial low-quality data participation. It employs the DCEM model to evaluate data quality and the SAP protocol for secure interactions. Experimental results on the MNIST dataset show that PPFL-LQDP

outperforms existing methods like SecProbe, PPFDL, and EPPFL in accuracy and robustness. Its single-cloud architecture ensures scalability and usability, making it well-suited for real-world applications.

In their study titled “A Practical Privacy-Preserving Federated Learning Algorithm Based on Local Federalization Scheme,” Yong Li et al.[7] The LF3PFL framework enhances privacy in federated learning through localized updates, preserving data confidentiality without sacrificing accuracy. It employs cross-entropy optimization and fine-tuning to minimize information loss. Evaluated on CIFAR-10, MNIST, and Shakespeare datasets, it uses five local models: Simple-CNN, ModerateCNN, Lenet, VGG9, and ResNet18. Compared to FedAvg, FedProx, FedMA, DP-FL, Chain-PPFL, and CS-FL, LF3PFL achieves superior accuracy: 81.63% on CIFAR-10, 97.21% on MNIST, and 55.86% on Shakespeare. The framework demonstrates robust performance in privacy-preserving FL.

In their study titled “Security of Federated Learning: Attacks, Defensive Mechanisms, and Challenges,” Mourad Benmalek et al.[8] Federated Learning (FL) is a decentralized machine learning technique that aggregates local updates while maintaining privacy to train a global model. FL is susceptible to attacks such as poisoning, inference, communication, and free-riding, notwithstanding its benefits. Differential Privacy (DP), Secure Multi-party Computation (SMC), anomaly detection, resilient aggregation, and federated distillation are examples of defense techniques. Nevertheless, these approaches have drawbacks that make striking a balance between security, privacy, and performance difficult. These changing difficulties require more investigation.

In their study titled “Balancing privacy and performance in federated learning: A systematic literature review on methods and metrics,” Samaneh Mohammadi et al. [9] In order to balance privacy with performance measures like accuracy and overhead, this work examines privacy-preserving techniques in Federated Learning (FL). It divides privacy strategies into four categories: blockchain-based, encryption-based, perturbation-based, and hybrid approaches. Issues like higher communication and processing overhead are resolved. For global model updates, aggregation algorithms and data partitioning techniques (horizontal, vertical, and transfer FL) are covered. In order to improve FL privacy and performance, the paper looks at previous

developments, addresses unresolved issues, and makes recommendations for future research areas.

In their study titled “An Optimized Differential Privacy Enabled Privacy Preserving Federated Learning,” Maria Iqbal et al.[10] This work suggests using an Optimized Differential Privacy (ODP) strategy in Federated Learning (FL) to improve data privacy by adjusting delta ($1e-5$, $1e-3$) and noise ($\sigma = 1.2, 2.0, 4.0, 8.0$) values while maintaining a constant epsilon ($\epsilon = 0.5$). The approach shows 90.5% accuracy in the non-IID scenario and 98% accuracy in the IID setting when validated using an MLP model on the MNIST dataset. When compared to regular FL, the optimized DP phase exhibits higher accuracy, successfully striking a balance between model training efficiency and privacy preservation.

In their study titled “Communication-Efficient Learning of Deep Networks from Decentralized Data,” H. Brendan McMahan et al.[11] Decentralized model training is made possible by Federated Learning (FL), which both lowers communication costs and protects privacy. By combining local updates, the Federated Averaging (FedAvg) method outperforms Federated Stochastic Gradient Descent (FedSGD) in terms of convergence speed. FedAvg can withstand imbalanced and non-IID data. FedAvg's efficacy was demonstrated by experiments on MNIST, CIFAR-10, and a language modeling challenge. It beat FedSGD (99.22%) with an accuracy of 99.44% on MNIST. After 2,000 rounds, FedAvg achieved 85% accuracy on CIFAR-10, greatly lowering communication.

In their study titled “FL-APB: Balancing Privacy Protection and Performance Optimization for Adversarial Training in Federated Learning,” Liu, Wu et al.[12] The FL-APB technique uses adversarial training to enhance model privacy and performance. It outperforms FedAvg utilizing LeNet for image classification when tested on CIFAR-10 with non-IID data. FL-APB utilizes Adam with a rate of 3×10^{-4} , whereas FedAvg uses SGD with a learning rate of 0.15. Using adversarial training techniques such as FGSM, PGD, MIFGSM, and TPGD, privacy attack resistance is assessed using DLG and iDLG. The accuracy of FL-APB is higher (48.88%) than that of the baseline (47.68%). Additionally, it exhibits greater resilience in a range of hostile environments.

In their study titled “Advancements in Privacy-Preserving Techniques for Federated Learning: A Machine Learning Perspective,” Monika Dhananjay Rokade et al.[13] Federated Learning (FL) keeps data decentralized, allowing collaborative machine learning while protecting data privacy. Privacy issues still exist, though, because model updates have the potential to expose private data. Model changes are protected by methods such as homomorphic encryption, differential privacy, trusted execution environments (TEEs), and secure multi-party computing (MPC). These techniques provide secure aggregation, data confidentiality, and regulatory compliance. To address scalability, performance overhead, and robustness while preserving privacy and stakeholder trust, more study is required.

In their study titled “Personalized Privacy-Preserving Federated Learning,” Cédric Boscher et al.[14] DINAR is a privacy-preserving federated learning middleware that defends against membership inference attacks (MIAs) without sacrificing computational overhead or model correctness. Unlike approaches that require heavy cryptography or disturb all layers, it protects only the most sensitive model layers. It eschews the significant overhead associated with differential privacy and cryptography. Even in non-IID environments, DINAR offers optimal anonymity while preserving model utility.

In their study titled “A Hybrid Approach to Privacy-Preserving Federated Learning,” Stacey Truex et al.[15] Secure Multiparty Computation (SMC) and Differential Privacy (DP) are combined in a federated learning technique to improve model accuracy while maintaining robust privacy guarantees. While DP reduces inference risks and balances scalability and noise injection, SMC protects message exchanges. Among the models supported by the system are SVMs, CNNs, and Decision Trees. It demonstrated excellent accuracy and no privacy loss when tested on the Nursery, MNIST, and Gisette datasets. In terms of accuracy and privacy protection, the method fared better than current FL Techniques.

Table.1 Literature Survey

S. No	Title & Authors	Year & Published	Approach	Outcome	Limitations
1.	Personalized Privacy-Preserving Framework for Cross-Silo Federated Learning- Van-Tuan Tran , Huy-Hieu Pham	2024, IEEE Transactions on Emerging Topics Computing	PPFPL integrating DP, GANs, and meta-learning	96.4% (MNIST), 91.99% (F-MNIST), 80% (CIFAR-10), 48% (CIFAR-100)accessibility	Increased complexity due to multiple techniques
2.	Federated Learning with Differential Privacy- Adrien Banse*, Jan Kreischer, Xavier Oliva I Jurgens	2023, IEEE Transactions on Information Forensics and Security	RDP-PFL using RDP-SGD with convergence bounds	96.4% (MNIST), 80% (CIFAR-10)control	Requires dynamic budget tuning; theoretical complexity
3.	Towards Adaptive Privacy Protection for Interpretable Federated Learning- Zhe Li,	2024, IEEE Transactions on Mobile Computing	AGP with selective DP noise and interpretability	High accuracy on MNIST, Blood, and Animals (IID & Non-IID)	Reduced performance in Non-IID; model-specific
4.	Personalized Federated Learning With Differential Privacy and Convergence Guarantee- Kang Wei, Jun Li, Senior,	2023, IEEE TPAMI	FedAvg + DP (Opacus) on CNN, Logistic Regression	DP reduces accuracy significantly (MNIST: 95% $\rightarrow$ 75%)spread	Lower accuracy on non-IID/small datasets
5.	Design and implementation of privacy-preserving federated learning algorithm for consumer	2024, Alexandria Engineering Journal	CNN + DP + Blockchain for smart appliances	Better accuracy with ϵ -DP normalization	Limited to MNIST; ϵ -sensitive performance

S. No	Title & Authors	Year Published	Approac	Outcome	Limitations
6.	Privacy-preserving federated learning based on partial low-quality data- Huiyong Wang,Qi Wang, Yong Ding, Shijie Tang4 and Yujue Wang5	2024, Journal of Cloud ComputingResearch	Paillier crypto + DCEM + SAP protocol	Outperforms SecProbe, PPFDL, EPPFL on MNIST	Focus on low-quality data; tested on one dataset
7.	A Practical Privacy-Preserving Federated Learning Algorithm Based on Local Federalization Scheme- Yong Li, Gaochao Xu, Xutao	2024, Entropy	Local updates + cross-entropy + fine-tuning	97.21% (MNIST), 81.63% (CIFAR-10)	Dataset-dependent, complexity in fine-tuning
8.	Security of Federated Learning: Attacks, Defensive Mechanisms, and Challenges- Mourad Benmalek, Mohamed Ali	2022, Revue d'Intelligence Artificielle	Overview of FL attacks and defenses	Highlights limitations in each technique	No novel solution proposed
9.	Balancing privacy and performance in federated learning: A systematic literature review on methods and metrics- Samaneh Mohammadi, Ali Balador	2024, J. of Parallel and Distributed Computing	Categorization: encryption, perturbation, hybrid	Insights on trade-offs across methods	Generalized; lacks experimental validation
10.	An Optimized Differential Privacy Enabled Privacy Preserving Federated Learning- Maria Iqbal, Asadullah Tariq, Muhammad	2024, IEEE ACCESS	MLP + DP with variable noise/delta values	98% (IID), 90.5% (Non-IID) on MNIST	Accuracy depends heavily on DP parameters

S. No	Title & Authors	Year Published & Journal	Approach	Outcome	Limitations
11.	Communication-Efficient Learning of Deep Networks from Decentralized Data- H. Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson	2017, AISTATS	FedAvg vs. FedSGD	99.44% (MNIST), 85% (CIFAR-10)	Older method; not privacy-centric
12.	Balancing Privacy Protection and Performance Optimization for Adversarial Training in Federated Learning- Liu, Wu,	2024, Electronics	FL + adversarial training (FGSM, PGD)	48.88% vs. 47.68% (CIFAR-10)	Limited gain; adversarial methods add complexity
13.	Advancements in Privacy-Preserving Techniques for Federated Learning: A Machine Learning Perspective- Monika Dhananjay Rokade, Suruchi	2024, J. of Electrical Systems	FL + DP, HE, MPC, TEEs	Secure aggregation with regulatory focus	High computational overhead
14.	Personalized Privacy-Preserving Federated Learning- Cédric Böscher, Nawel Benarba, Fatima Elhattab, Sara Bouchenak	2024, ACM	Selective layer obfuscation for MIAs	Reduced MIA success rate; high utility	May not protect all forms of attacks
15.	A Hybrid Approach to Privacy-Preserving Federated Learning- Stacey Truex, Georgia Nathalie Baracaldo, Ali Anwar, Thomas, Heiko, Rui Zhang, Yi Zhou	2019, arXivWorkshops	FL with ID3, CNN, SVM + DP + SMC	High accuracy, low privacy loss	May face scalability in real-time apps

2.2 EXISTING SYSTEM

The existing system for Federated Learning (FL) primarily relies on standard FL frameworks that implement privacy-preserving techniques, such as Differential Privacy (DP) and Secure Aggregation. In this setup, multiple clients collaboratively train a shared global model without directly sharing their raw data. Instead, only model updates (gradients) are exchanged between clients and the central server, ensuring a degree of privacy. However, despite these privacy-preserving measures, the system is still vulnerable to certain privacy threats and performance limitations.

One of the key methodologies used in the existing system is DP-based FL, where random noise is added to model updates before they are shared with the central server. This approach helps in protecting user data from being reconstructed through gradient inversion techniques. However, this added noise leads to a trade-off between privacy and accuracy—while strong privacy settings can reduce the risk of data leakage, they also degrade the model’s overall performance.

Another major limitation of the existing system is model divergence due to data heterogeneity. In a cross-silo federated learning setting, different clients often have non-identically distributed (Non-IID) data, causing inconsistent model training and suboptimal generalization. The lack of proper personalization mechanisms in existing FL frameworks makes it challenging to adapt the global model to individual clients’ data distributions, resulting in reduced performance for diverse datasets.

Additionally, the existing system faces communication inefficiencies, as model updates must be frequently exchanged between the clients and the central server. This results in high bandwidth consumption and increased latency, making the training process computationally expensive, especially for large-scale datasets. In terms of empirical results, the existing system has achieved 0.94 accuracy on the MNIST dataset and 0.86 accuracy on the FMNIST dataset when using Differential Privacy. While these results indicate that the system maintains reasonable accuracy while incorporating DP-based privacy protection, the overall performance is still lower than expected due to the inherent trade-offs of DP.

Overall, while the existing system offers fundamental privacy protection in federated learning, it suffers from privacy leakage risks, communication inefficiencies, model divergence, and limited personalization capabilities, creating a strong need for an improved framework. The proposed system aims to address these limitations by introducing personalized model learning, optimized communication protocols, and better privacy-preserving techniques, ensuring higher accuracy and better performance without compromising user data security.

2.3 DISADVANTAGES OF EXISTING SYSTEM

While the existing Federated Learning (FL) framework with Differential Privacy (DP) offers fundamental privacy protection, it still suffers from several limitations that impact both performance and usability. These disadvantages highlight the need for a more efficient and personalized approach to federated learning.

1. Privacy Leakage Risks: Although DP is employed to add noise to model updates and protect user data, it is not entirely foolproof. Sophisticated gradient inversion techniques and adversarial attacks can still infer sensitive information from shared model updates, leading to potential privacy breaches. Additionally, fine-tuning DP parameters to achieve an optimal balance between privacy and accuracy remains a challenging task.

2. Accuracy-Privacy Trade-off: The introduction of DP significantly impacts model performance. Since DP adds noise to the gradients before aggregation, it reduces the risk of data reconstruction but at the cost of degraded model accuracy. In the existing system, MNIST achieves an accuracy of 0.94 and FMNIST achieves 0.86 under DP constraints, which is lower than what could be achieved without DP. This compromise makes it difficult to maintain both strong privacy and high model accuracy simultaneously.

3. Model Divergence Across Clients: The existing system struggles with data heterogeneity among clients. Since each client's local dataset can be vastly different (Non-IID data), the global model may fail to generalize effectively across all clients. This results in inconsistent learning and suboptimal performance for certain users, especially in real-world applications where personalized predictions are crucial.

4. High Communication Overhead: FL requires frequent exchanges of model updates between clients and the central server. This process demands high bandwidth and computational resources, making it inefficient in environments with network constraints. Additionally, as the number of participating clients increases, the communication overhead scales up, leading to delays in model convergence and increased latency.

5. Computational and Energy Costs: Since FL relies on edge devices (such as mobile phones, IoT devices, or local servers) for model training, the computational burden on individual clients can be high, especially for complex deep learning models. The existing system does not optimize energy consumption, making it less feasible for devices with limited processing power and battery life.

6. Scalability Challenges: As the number of clients in the FL network grows, maintaining privacy, efficiency, and model performance becomes increasingly difficult. The current system does not have effective mechanisms for handling large-scale FL networks, leading to bottlenecks in aggregation, increased latency, and potential failure in synchronization among participating clients.

2.4 PROPOSED SYSTEM

To address the limitations of the existing Federated Learning (FL) system, we propose a Privacy-Preserving Federated Learning Framework that enhances model performance, efficiency, and security. The proposed system introduces personalized model adaptation techniques, optimized privacy-preserving mechanisms, and improved FL parameter tuning to achieve better accuracy and communication efficiency while maintaining strong privacy guarantees. The core of our approach is a Convolutional Neural Network (CNN), which is well-suited for image classification tasks and enables efficient feature extraction. By integrating CNN into the FL framework, the proposed system can better capture spatial hierarchies in data, improve learning efficiency, and enhance model performance compared to traditional approaches.

Let's look into the Key Enhancements in the Proposed System:

1. Improved Accuracy with Optimized FL Parameters

The existing system achieved 0.94 accuracy on MNIST and 0.86 on FMNIST using Differential Privacy (DP). However, the proposed system fine-tunes FL parameters to enhance accuracy while maintaining privacy protection. With RDP and optimized FL parameters, we achieve a accuracy of 0.96 on MNIST, and 0.82 on FMNIST surpassing the previous DP-constrained model for MNIST while ensuring privacy. This improvement is achieved by adjusting key FL hyperparameters such as learning rate (lr), local epoch count (E), client sampling rate (C), and batch size, which allow for better model generalization and convergence.

2. Federated Learning for Client-Specific Adaptation

A major limitation of the existing system is that it applies a single global model to all clients, which fails to account for heterogeneous (non-IID) data distributions among different participants. Our proposed system incorporates local fine-tuning techniques, where each client receives a base model and further adapts it based on its local dataset. This approach ensures:

Better generalization to client-specific data. Higher accuracy on personalized tasks. Reduced model divergence, allowing each participant to train an optimized version of the model suited to its data characteristics.

3. Enhanced Privacy-Preserving Mechanisms

While DP is retained as a core privacy mechanism, the proposed system modifies the application of differential privacy by fine-tuning key parameters such as:

Epsilon (ϵ): Adjusted to 4.0 for optimal balance between privacy and model utility.

Delta (δ): Set to $1e-5$, ensuring strong privacy guarantees.

Gradient clipping (clip): Set to 0.1, preventing extreme gradient values from compromising privacy. These adjustments minimize the noise added to model updates while maintaining a high level of privacy, overcoming the privacy-accuracy trade-off in the existing system.

4. Customized CNN Architecture for FL

Instead of using generic deep learning models, the proposed system specifically employs a custom CNN for federated training. CNN's architecture allows for efficient feature extraction using convolutional layers. Better learning from spatial hierarchies in image-based datasets like MNIST and FMNIST. Reduced computational burden on clients due to CNN's ability to learn hierarchical representations with fewer parameters. This results in faster convergence, improved accuracy, and lower computational overhead, making the FL system more practical for resource-limited edge devices.

5. Reduced Communication Overhead and Bandwidth Usage

A significant improvement in the proposed system is the optimization of communication efficiency in FL. The existing system suffers from high network traffic due to frequent model updates, which slows down training and increases costs. Our proposed system reduces the number of model update exchanges while maintaining performance by:

Reducing the total number of training rounds ($\text{tot_T} = 10$), leading to faster convergence.

These improvements make the FL framework more scalable, allowing it to handle large-scale deployments with lower resource demands.

2.5 CONCLUSION

The literature survey provides a comprehensive analysis of various privacy-preserving techniques in Federated Learning (FL), highlighting their advantages and limitations. Several studies integrate Differential Privacy (DP), Secure Multi-Party Computation (SMC), cryptographic methods, and adversarial training to enhance security and performance in FL systems. While these methods improve privacy protection, they also introduce trade-offs in model accuracy, convergence speed, and computational overhead. The survey emphasizes the need for a balanced approach that ensures data confidentiality without compromising efficiency.

Additionally, the review of existing systems has demonstrated that while FL reduces privacy risks compared to traditional centralized learning, challenges such as

gradient leakage, high communication costs, and lack of effective personalization techniques remain significant obstacles. Studies have also shown that DP-based FL frameworks, despite their enhanced privacy mechanisms, suffer from performance degradation due to added noise, leading to lower accuracy in certain applications.

Furthermore, the literature highlights the necessity of developing frameworks that adapt to heterogeneous data distributions, optimize communication efficiency, and improve security without excessive computational burden. Researchers have explored innovative techniques such as adaptive privacy budgets, personalized FL models, and optimized aggregation mechanisms, but there is still a need for improved frameworks that can be efficiently deployed in real-world scenarios.

By analyzing various privacy-preserving FL methodologies, this survey lays the groundwork for the development of a Privacy-Preserving Federated Learning Framework that addresses the identified limitations. The insights gained from this survey guide the proposed system's design, ensuring it achieves a balance between privacy, model performance, and computational feasibility while overcoming existing challenges in federated learning environments

3.REQUIREMENT ANALYSIS

3.1 INTRODUCTION

Requirement analysis is a crucial phase in the software development lifecycle that helps define the system's functional and non-functional needs. It serves as the foundation for designing and implementing the project efficiently, ensuring that the developed system meets user expectations and technical constraints. For this project, the Privacy-Preserving Federated Learning Framework, requirement analysis involves understanding the privacy concerns, computational constraints, and data communication challenges associated with federated learning.

In traditional machine learning, centralized data collection poses significant privacy risks, leading to strict regulatory requirements such as GDPR . Federated Learning (FL) addresses this issue by training models locally without sharing raw data. However, FL introduces new challenges like communication overhead, security threats, and inconsistencies in model updates across different data silos. Therefore, it is essential to analyze the software, hardware, and user requirements to develop a framework that ensures efficient, scalable, and privacy-preserving federated learning.

This chapter outlines the necessary software and hardware requirements, along with a detailed specification of user expectations and system capabilities. Additionally, a content diagram will illustrate the overall project structure, followed by an analysis of the key algorithms and flowcharts used to implement the proposed framework. By systematically defining these requirements, this project ensures that the solution is robust, secure, and practical for real-world applications.

3.2 SOFTWARE REQUIREMENT SPECIFICATION

Software Requirement Specification (SRS) serves as a structured document that outlines the essential functional and nonfunctional requirements of a system. It acts as a blueprint for developers, ensuring that all stakeholders understand the project's scope, constraints, and objectives. The success of any software project depends on clearly defining what the system should do and the necessary infrastructure to support it.

For this project, the Privacy-Preserving Federated Learning Framework, the SRS focuses on defining requirements related to user expectations, software dependencies, and hardware capabilities. Given the complexity of federated learning, it is essential to ensure that the system supports secure communication, efficient data handling, scalable model training, and privacy techniques. The following sections provide an in-depth analysis of the user, software, and hardware requirements essential for the implementation of this project.

3.2.1 USER REQUIREMENTS

User requirements define the expectations of individuals or organizations interacting with the system. Since this project involves federated learning, the primary users include data owners (clients), model trainers (aggregators), and system administrators. These users have specific needs that must be met to ensure seamless functionality, security, and personalization.

The key user requirements for this system include:

Privacy-Preserving Model Training: Users should be able to train machine learning models locally without exposing their raw data to external servers. This ensures compliance with data protection regulations and prevents unauthorized access.

Seamless Data Handling: The system should enable secure data processing and encryption mechanisms to prevent privacy breaches during model training and updates.

Real-time Model Synchronization: Since FL involves multiple participants training models independently, users should experience efficient and real-time model synchronization with minimal latency.

User-friendly Interface: The system should provide an intuitive interface for monitoring model performance, checking accuracy trends, and applying privacy settings.

Performance Monitoring: Users should be able to track the efficiency and accuracy of their locally trained models and compare them with global performance metrics.

These requirements ensure that the system aligns with the needs of data owners, researchers, and developers involved in federated learning model training.

3.2.2 SOFTWARE REQUIREMENTS

Table.2 Software Requirements

Requirement Specification	Software Requirements
Programming Language	Python(Python3)
Machine Learning Libraries	Torchvision, PyTorch, kymatio
Privacy-Preserving Tools	Using Manual differential privacy accounting
Storage Solutions	Google Drive
Development Environment	Jupyter Notebook , Google colab, VS Code
Operating System	Windows 11 Home

The software requirements define the tools, frameworks, and libraries necessary for developing and deploying the Privacy-Preserving Federated Learning Framework. This includes programming languages, machine learning libraries, privacy-enhancing tools, and database management systems that will be used in the project.

Programming Languages:

Python – Primary language for implementing machine learning models and federated learning algorithms.

JavaScript (Optional) – If a web-based monitoring interface is required.

Machine Learning Libraries:

Torchvision / PyTorch – Used for building and training deep learning models.

kymatio – For implementing standard Scatter CNN model and to add wavelet transforms in the model.

Federated Learning Frameworks:

Used RDP accounting and calibrating noise efficiently by doing the computations normally.

Privacy-Preserving Tools:

Opacus – Used to integrate Differential Privacy (DP) techniques into model training.

Development Environment:

Jupyter Notebook / VS Code – IDEs for writing, testing, and debugging code.

3.2.3 HARDWARE REQUIREMENTS

Table.3 Hardware Requirements

Requirement Specification	Hardware Requirements
Processor	Intel Core i5 or i7
Memory (RAM)	16GB RAM
Storage	SSD with a minimum of 512GB
Network	100 Mbps
GPU	NVIDIA A100(local) / Tesla V100

Since federated learning involves computationally intensive model training and secure communication, selecting appropriate hardware is critical.

Processor: Intel Core i5 or Intel Core i7

Memory (RAM): At least 16GB RAM to handle data preprocessing and training tasks efficiently.

Storage: SSD with a minimum of 512GB to store datasets, model checkpoints, and logs.

Network: Stable internet connection (at least 100 Mbps) for seamless model update exchanges.

GPU (Recommended for Large Models): NVIDIA A100 - if local system is used else Tesla V100 for optimizing federated model computations.

3.3 CONTENT DIAGRAM OF PROJECT

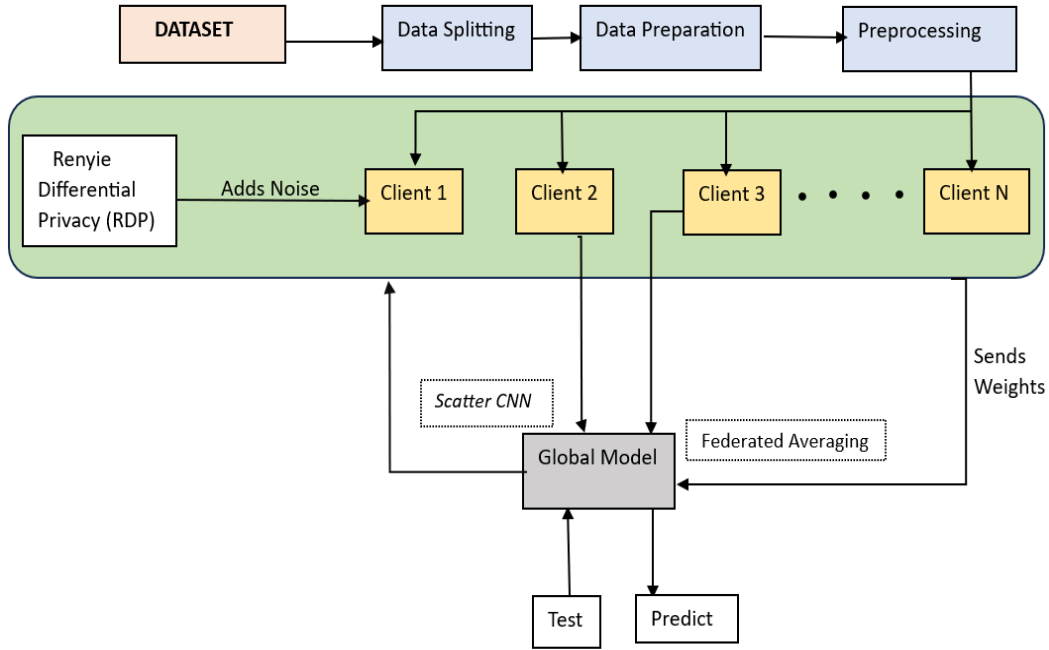


Fig 3.1 Content Diagram

Fig 3.1 represents the Federated Learning (FL) with Renyie Differential Privacy (RDP) using scatter CNN-based Image Classification for **MNIST** and **FMNIST** datasets. The architecture ensures data privacy and decentralized learning by training models locally on multiple clients and aggregating their results into a global model. Below is the step-by-step breakdown:

1. Dataset Preparation

- The MNIST or FMNIST dataset is first acquired.
- It undergoes data splitting, where the dataset is distributed among multiple clients instead of a central server.
- Each client receives its own portion of the dataset.
- Preprocessing ensures images are normalized, resized, and structured for CNN input

2. Federated Learning Setup

- Instead of sending raw data to a central server, each client trains a local model using Scatter-CNN.
- Differential Privacy (DP) is applied to ensure security, where noise is added before training.
- The trained model weights are then sent to the central global model.

3. Local Model Training on Clients

- Each client (Client 1, Client 2, ... Client N) trains its CNN model independently.
- Differential privacy techniques, such as Rényi Differential Privacy (RDP) and Gaussian Noise, are used to protect sensitive data.
- After local training, only model updates (weights) are sent to the global model, not the actual dataset.

4. Global Model Aggregation

- The Global Model collects and aggregates the model weights from all clients.
- The aggregation is done using Federated Averaging (FedAvg), which optimally combines updates from all clients to improve the model's accuracy.
- Scatter CNN ensures efficient distribution of training tasks across clients.

5. Model Testing & Prediction

- Once the global model is trained, it undergoes testing with unseen data.
- Finally, it is used for prediction, where it classifies new images uploaded by the user as either digits (MNIST) or fashion items (FMNIST).

3.4 ALGORITHMS AND FLOWCHARTS

ALGORITHMS:

This project investigates numerous machine learning and deep learning algorithms for image classification on the MNIST and FashionMNIST datasets. The algorithms were deployed inside a Federated Learning (FL) environment where data is kept distributed on client devices, while model updates are exchanged. Differential privacy was applied to boost security, and ScatterLinear was the best-performing model eventually. A full explanation of all utilized and tested algorithms is presented below.

1. Logistic Regression

Logistic Regression is a simple linear classification model implemented as a baseline model for this project. It directly maps input image pixels to output class probabilities via a softmax activation function. In the case of MNIST and FashionMNIST, images are flattened to a vector of pixel values and then run through a single layer with 10 output units (one for every class).

In Federated Learning, logistic regression involves extremely low computational and communication overhead. It cannot learn complex features, though, so it does not work for extracting complex patterns from image data. It contributed to setting the standard for the comparison of deeper and more intricate models' performance.

2. Multilayer Perceptron (MLP)

MLP is a generalization of logistic regression with one or more hidden layers. For this project, a basic MLP with a single hidden layer was employed to enhance learning capacity. The MLP consists of:

- An input layer (flattened image vector)

- A hidden layer with nonlinear activation (ReLU)

- An output layer with softmax activation

This model learns mid-level representations which enhance classification precision over logistic regression. MLPs also work moderately well under federated conditions but can overfit when training local datasets that are highly imbalanced unless regularized.

3. Three-Layer MLP

The 3-layer MLP extends the standard MLP by adding two hidden layers. This deeper architecture enhances the network's capacity to model complex, non-linear interactions in image data. Each hidden layer contains ReLU activation to add non-linearity.

In the FL setting, the 3-layer MLP also performed better than the standard MLP, particularly on FashionMNIST, which is visually more complex. But its increased number of parameters needed more privacy-preserving noise to be added in the use of Differential Privacy, which had a slight effect on performance.

4. Simple CNN

CNNs are particularly suitable for image classification because they can extract local spatial features. The CNN architecture employed in this project consists of:

Convolutional layers to extract local patterns

Pooling layers to downsample spatial dimensions

Fully connected layers for the final classification

This CNN model was experimented with 1, 5, and 10 clients under federated learning, which performed well on both the MNIST and FMNIST datasets. CNNs are less sensitive to image transformations and result in much better performance compared to MLPs. They consume more bandwidth during weight updates and training time.

5. ScatterLinear Transform(Model used in our project)

The ScatterLinear model combines the Scattering Transform (through the Kymatio library) with an ordinary linear classifier. Contrary to CNNs, which learn convolutional filters, the scattering transform learns by fixed wavelets invariant and stable image features.

Main features:

Scattering Transform learns dense, low-variance representations of the input images.

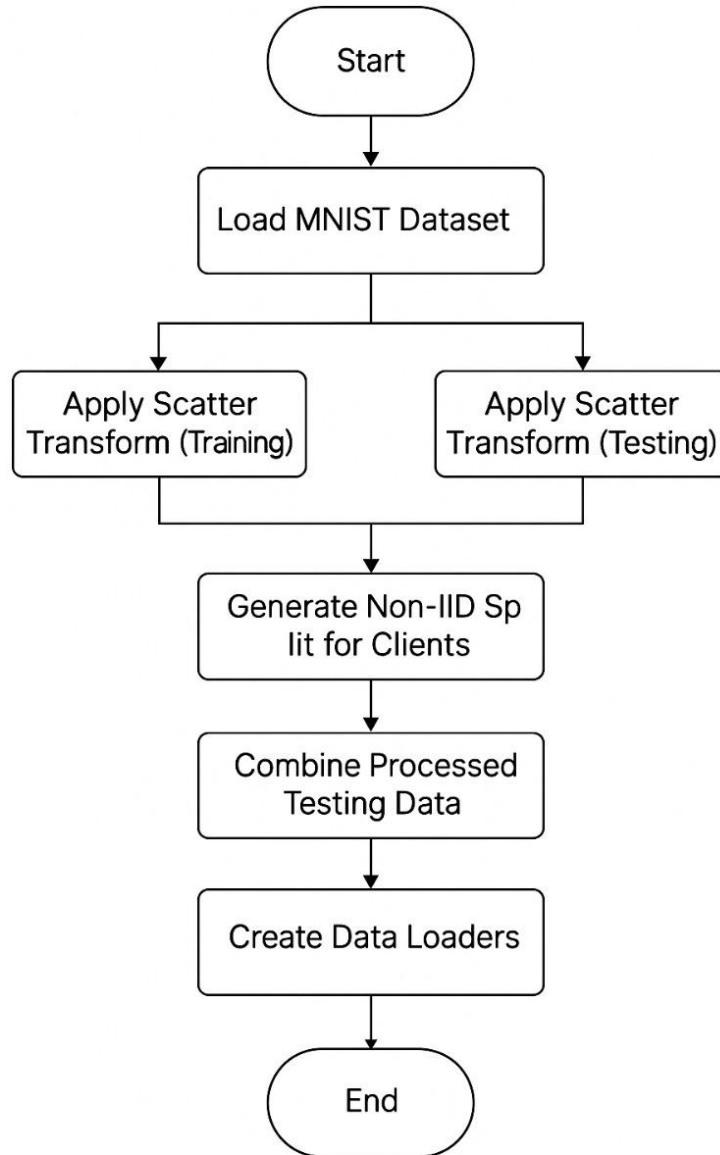
These features are then processed using a Linear Classifier for prediction.

The model is much smaller with fewer parameters, which makes it well-suited for Differential Privacy since smaller updates need less noise to ensure privacy.

ScatterLinear worked best in federated environments because of its trade-off between accuracy, efficiency, and privacy. It is extremely robust to client data heterogeneity and has very low training time requirements, making it the best fit for the final model.

FLOWCHARTS:

1.Data Extraction:



Data Extraction Process for Federated

Fig 3.2 Data Extraction

MNIST Dataset: The dataset is loaded using `torchvision.datasets.MNIST`.

Scatter Transform: This transformation helps extract features that could be more useful for training. The `get_scatter_transform()` function applies this transformation.

Non-IID Split: This step simulates a real-world scenario where each client has data that is not identically distributed. The data is split by labels into different sets that are used by different clients in Federated Learning.

Data Loaders: Data is loaded in batches for training and testing. This allows clients to work on small portions of data locally.

2. Data Preprocessing :

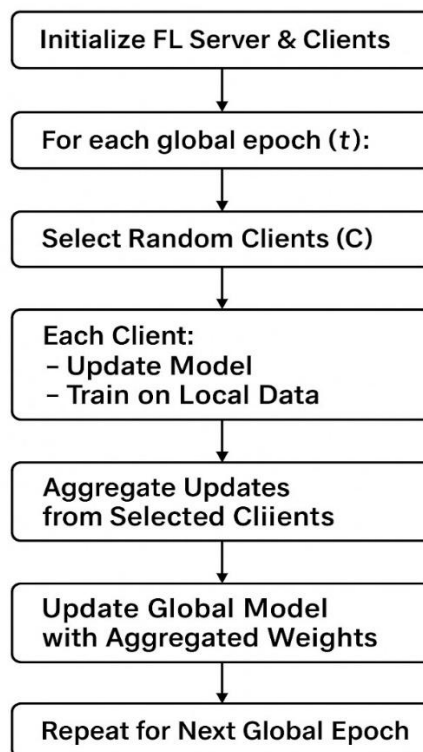


Fig 3.3 Data Preprocessing

1.Initialize FL Server & Clients: The Federated Learning server and clients are initialized. The server holds the global model, while each client has its local model and data.

2. For Each Global Epoch (t): The training process iterates over a specified number of global epochs. Each epoch represents a round of training where the global model is updated based on client contributions.

3. Select Random Clients (C): A random subset of clients is selected for the current training round. This selection is crucial for simulating a federated learning environment where not all clients participate in every round.

4. Each Client: Update Model & Train on Local Data: Each selected client updates its local model based on its local data. This involves training the model on the client's non-IID dataset for a specified number of local epochs.

5. Aggregate Updates from Selected Clients: After local training, the clients send their model updates (weights) back to the server. The server aggregates these updates to form a new global model. This can be done using methods like averaging the weights.

6. Update Global Model with Aggregated Weights: The global model is updated with the aggregated weights from the clients. This new model reflects the knowledge gained from the participating clients.

7. Repeat for Next Global Epoch: The process repeats for the next global epoch until the specified number of epochs is reached.

3. Federated Learning System :

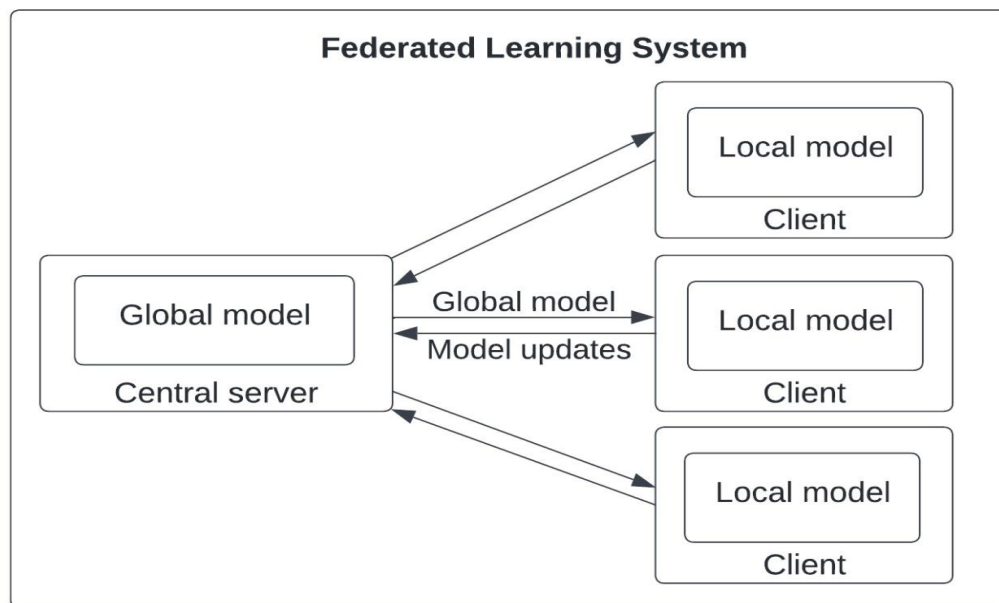


Fig. 3.4 Federated Learning System

4. Differential Privacy

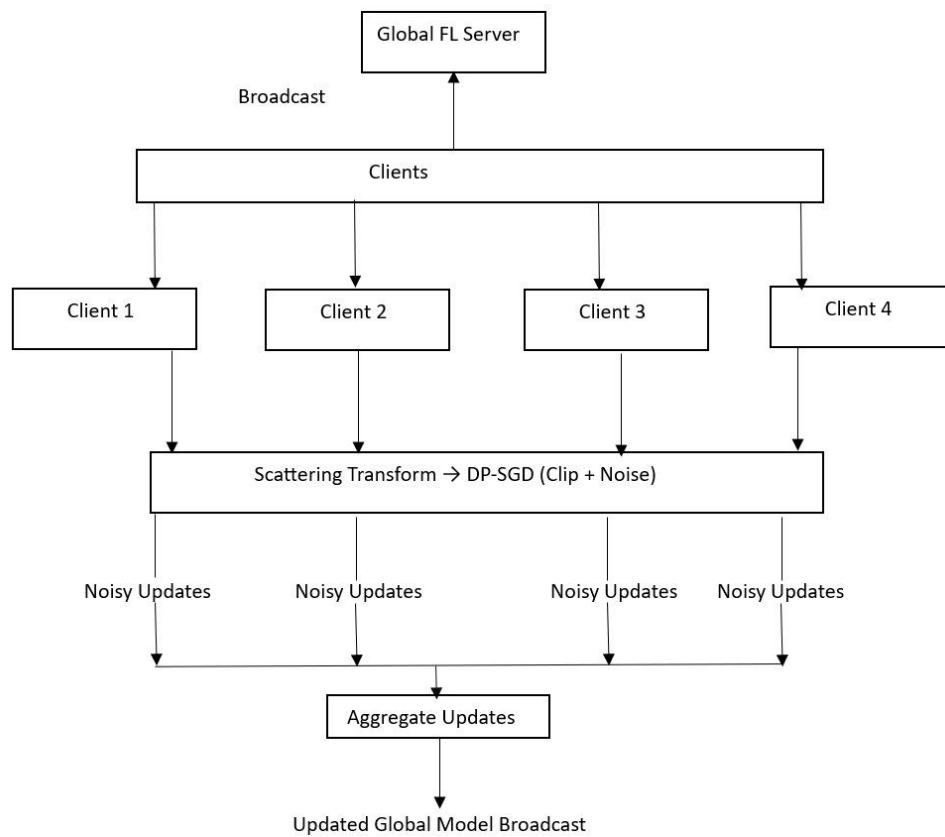


Fig 3.5 Differential Privacy Flowchart

Differential privacy (DP) and federated learning (FL) are distinct but related concepts in privacy-preserving machine learning. DP provides a formal framework for quantifying the privacy provided by an algorithm, while FL is a distributed training paradigm where models are trained across multiple devices or servers without sharing raw data. DP can be used to enhance the privacy guarantees of FL by adding noise to model updates or data before they are shared, making it harder to reconstruct individual data points.

5. Scattering Transform:

This is a fixed feature extractor (no learning involved).

It's especially useful for datasets like MNIST where translation-invariance and local structure matter.

Reduces the learning burden on the neural network.

Neural Network:

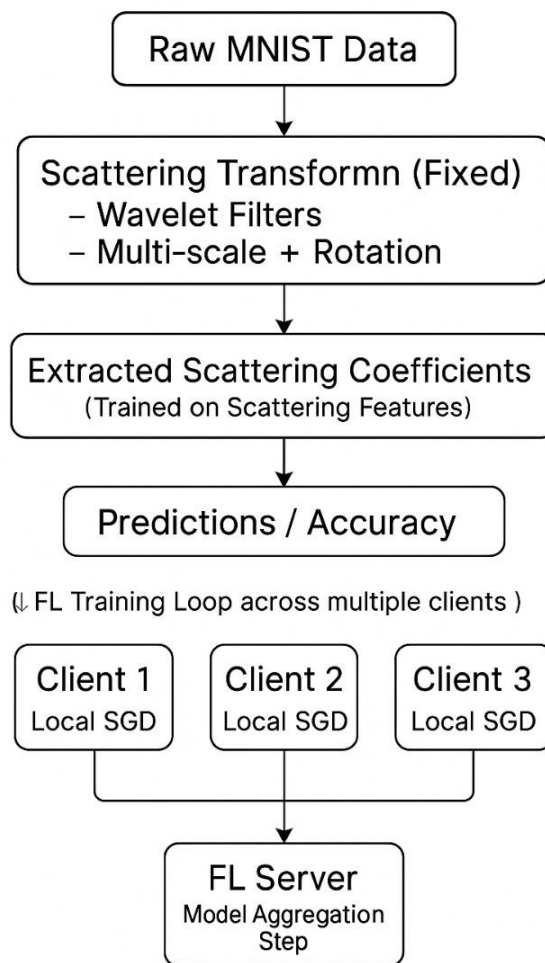
A shallow CNN is used after scattering.

Since scattering produces rich, structured features, even a small model can perform well.

Federated Learning Loop:

Each client uses local data (transformed with scattering) to train.

The server aggregates weights or gradients and updates the global model.



Scatter-Based Federated Learning Architecture

Fig 3.6 Scattering Transform Flowchart

3.5 CONCLUSION

The requirements analysis ensures that the necessary hardware and software components are in place for efficient implementation. By defining clear specifications, including dataset handling, model training, and security measures, the system is designed for optimal performance, scalability, and privacy preservation. This structured approach guarantees seamless integration and effective execution of the Federated Learning model.

4. DESIGN

4.1 INTRODUCTION

The design phase is a very important phase in the system development life cycle, as it is the foundation on which the entire system is constructed. It is a crucial phase that ensures the system is efficient, scalable, maintainable, and in line with technical and business needs. In this phase, the architecture of the system is carefully planned, and detailed specifications are created to inform the implementation process.

At its core, the design phase defines the structure and behavior of the system. It includes planning carefully how data moves through the system, how parts communicate with one another, and how the system will react to different inputs and situations. Unified Modeling Language (UML) Diagrams: UML diagrams like use case diagrams, class diagrams, sequence diagrams, and activity diagrams give a formal method of visualizing the structure of a system. They provide information about the static and dynamic features of the system.

A modular design methodology is prioritized at this stage. This involves dividing the system into autonomous, standalone modules or components. Each module performs an individual functionality and can be independently developed, tested, and sustained. This methodology improves:

- 1) Flexibility: New functionalities can be incorporated or altered without impacting the whole system.
- 2) Reusability: Modules can be reused across various sections of the application or in other applications.
- 3) Debugging and Testing: Bugs can be isolated and repaired more easily because of separation of concerns.

Additionally, a clear design provides for the fact that the system will be scalable in case user demand increases or needs arise in the future. It also ensures optimal performance by being able to optimize individual components. The design directly determines the usability of the system—enabling easy navigation and interaction—as well as its security, in terms of spotting and fixing vulnerabilities early on. The design stage takes conceptual requirements and converts them into a tangible blueprint for development. Through time and effort in this stage, developers can prevent expensive

rework, achieve seamless implementation, and develop a strong, user-focused system that fulfills its intended purposes with accuracy and dependability.

4.2 UML DIAGRAMS

4.2.1 USE CASE DIAGRAM

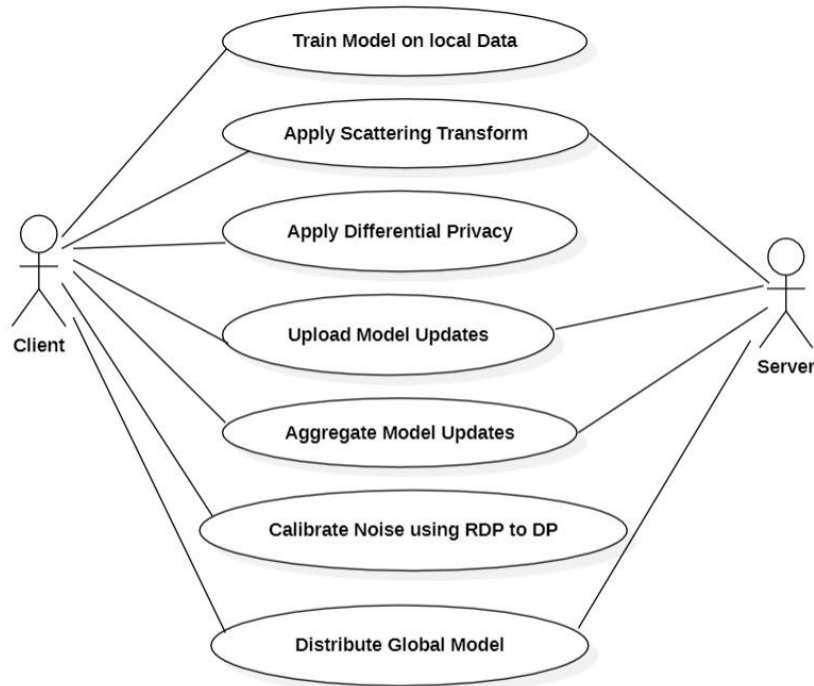


Fig 4.1 Use Case Diagram

Fig 4.1 illustrates the user and Federated Learning system interactions, outlining the primary actions the user can undertake and how the system acts upon them. The diagram depicts the flow of user-system interaction, guaranteeing usability, readability, and an improved understanding of system behavior.

Actors: Client, Server

Use Cases:

[Client] Train Model on Local Data

Description: The client trains a local model using its own private dataset.

[Client] Apply Scattering Transform

Description: Applies Scattering Transform on image data for robust feature extraction before training.

[Client] Apply Differential Privacy

Description: Adds Gaussian noise to local model updates using Differential Privacy mechanisms.

[Client] Upload Model Updates

Description: Sends differentially private model updates to the central server.

[Server] Aggregate Model Updates

Description: Aggregates the received local updates from clients using algorithms like FedAvg.

[Server] Calibrate Noise using RDP to DP

Description: Converts Rényi Differential Privacy (RDP) to (ϵ, δ) -DP for meaningful privacy guarantees.

[Server] Distribute Global Model

Description: Sends the updated global model to all clients for the next round of training

4.2.2 CLASS DIAGRAM

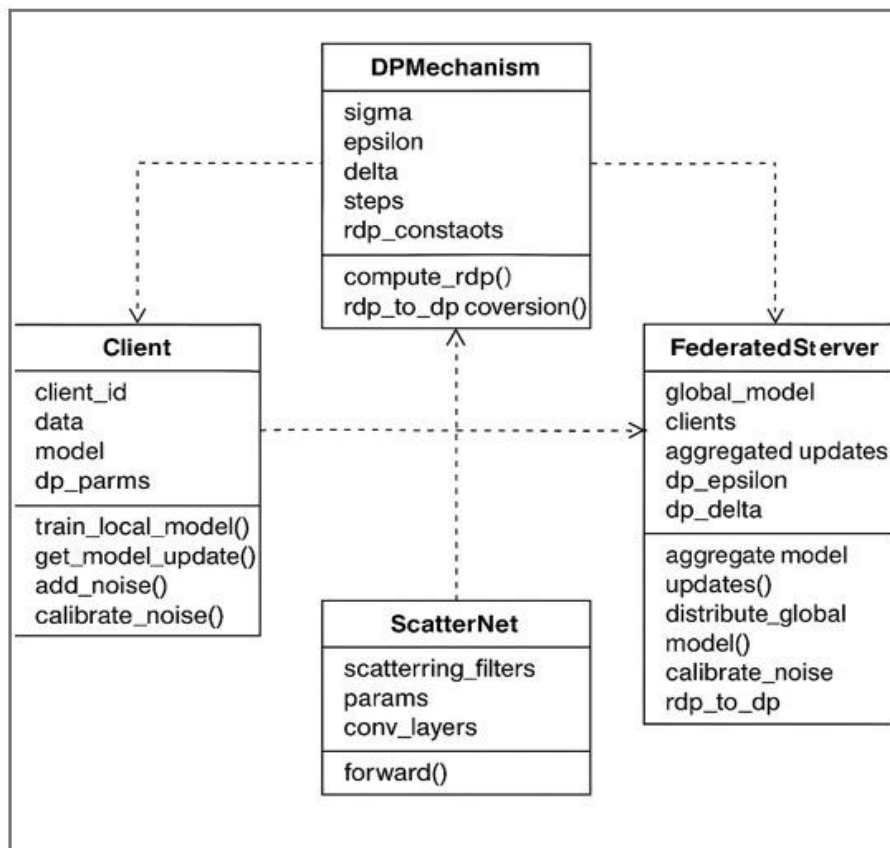


Fig 4.2 Class Diagram

Fig 4.2 describes

1. Client Class : Represents individual clients in the federated learning setup.

Attributes:

client_id: Unique identifier for the client.

data: Local data held by the client.

model: Local model (e.g., instance of ScatterNet).

dp_parms: Differential privacy parameters.

Methods:

train_local_model(): Trains the local model using the client's data.

get_model_update(): Retrieves model updates after training.

add_noise(): Adds Gaussian noise to the model updates for privacy.

calibrate_noise(): Uses RDP-based noise calibration to ensure differential privacy.

2. FederatedServer Class

Coordinates the federated learning process by aggregating updates and distributing the global model.

Attributes:

global_model: The central model being trained.

clients: List of participating clients.

aggregated_updates: Model updates collected from all clients.

dp_epsilon, dp_delta: DP parameters for privacy budget.

Methods:

aggregate_model_updates(): Aggregates noisy updates from all clients.

`distribute_global_model()`: Sends the global model to all clients.

`calibrate_noise()`: Works with `DPMecanism` to adjust noise levels.

`rdp_to_dp()`: Converts RDP measurements to (ϵ, δ) -DP.

3. DPMechanism Class

Handles all RDP (Rényi Differential Privacy) computations and conversions to standard differential privacy.

Attributes:

`sigma`: Noise scale.

`epsilon, delta`: Target privacy parameters.

`steps`: Number of training steps.

`rdp_constants`: Predefined constants for RDP calculation.

Methods:

`compute_rdp()`: Calculates RDP for a given noise scale and steps.

`rdp_to_dp_conversion()`: Converts RDP to (ϵ, δ) -DP.

4. ScatterNet Class

Custom deep learning model used by clients for training and evaluation.

Attributes:

`scattering_filters`: Filters used for wavelet-based scattering transformation.

`params`: Model hyperparameters.

`conv_layers`: Convolutional layers in the model.

Methods:

`forward()`: Forward pass of the model (used during training and inference).

apply_scattering(): Applies wavelet scattering transformation on input data.

train_step(): One training step using local data.

evaluate(): Evaluates the model on test data.

Relationships:

1. Client uses ScatterNet as the model.
2. Client and FederatedServer both rely on DPMechanism for computing and calibrating privacy noise.
3. FederatedServer manages communication and coordination among all Clients.

4.2.3 SEQUENCE DIAGRAM

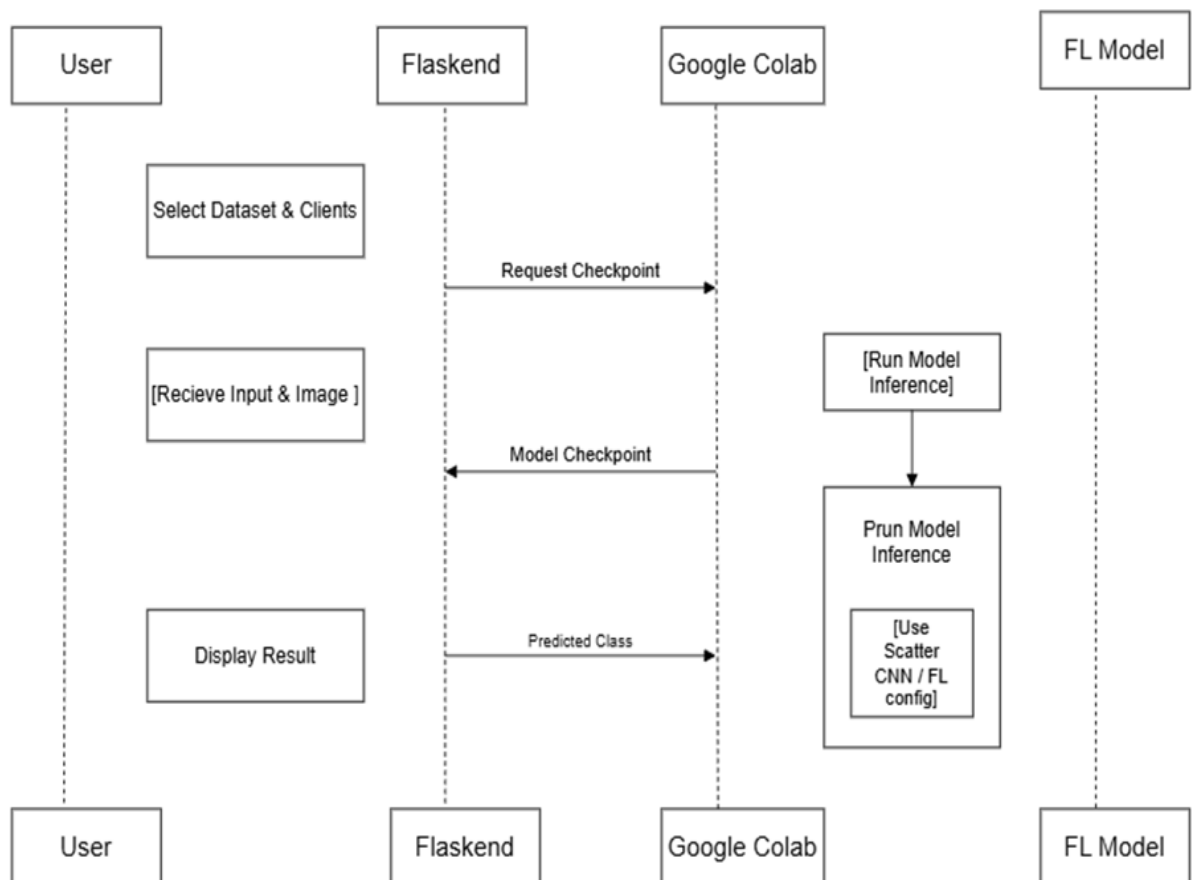


Fig 4.3 Sequence Diagram

Fig 4.3 illustrates how various elements within the system communicate with each other over time to carry out a classification operation with a federated learning (FL) model. The participating elements are: User (Frontend), Flaskend (Backend server implemented using Flask), Google Colab (where model inference occurs), FL Model (Pre-trained Federated Learning model).

User: Choose Dataset & Clients

- The user starts by choosing a dataset (e.g., MNIST, FMNIST) and indicating the number of federated clients to emulate.
- This input determines the setup for which model checkpoint to use.

User: Send Input & Image

- The user sends an image, which will be classified.
- The type of dataset, client size, and the image are forwarded to the Flask backend (Flaskend).

Flaskend: Request Checkpoint

- The backend leverages the configuration passed to make the correct model checkpoint request from Google Colab.
- This is to ensure that the model used for inference is suitable for the dataset and federated setup selected by the user.

Google Colab: Run Model Inference

- After receiving the request, Google Colab downloads the FL model and performs inference with the right configuration in the Scatter CNN (a CNN design suited for federated settings).
- The model has already bundled the learned knowledge of various decentralized clients.

Google Colab → Flaskend: Return Model Checkpoint

- Finally, Google Colab returns the model checkpoint or the result of the inference to the Flask backend.

Flaskend → User: Return Predicted Class

- Backend captures the output (i.e., the predicted class of the image uploaded) and returns it back to the user interface.

User views the result of the prediction shown.

- Asynchronous Interaction: The above figure depicts asynchronous messaging among components, particularly between Flask and Google Colab.

- Modular Design: Every component focuses on a separate responsibility:
- Frontend: Handling user input and display.
- Flaskend: Handling requests and communication of the result.
- Google Colab: Heavy computation as well as managing models.
- FL Model: Wrap the federated learning logic that has been trained.
- Cloud Offloading: Inference performed on Google Colab, offloading computation from heavy Flask backend and reducing inefficiencies.
- Clearness of Data Flow: Makes it easy to imagine data flowing from user to prediction and vice versa.
- Timing Insight: Illustrates the precise sequence of activities—helpful for debugging or optimizing performance.
- Component Communication: Indicates clearly how frontend, backend, cloud, and model layers communicate

4.2.4 ACTIVITY DIAGRAM

1. Start-The process begins.

2. Initialize Global Model on Server

The central server sets up a global model that will be trained collaboratively.

3. Distribute Global Model to All Clients

The initial model is sent to all clients (e.g., edge devices, hospitals, etc.) for local training.

[For Each Client]

These steps are repeated independently by each client:

4. Load Local Data

Each client loads its local dataset (which remains private and is not shared).

5. Initialize ScatterNet Model - Each client initializes a ScatterNet model architecture. ScatterNet is useful for extracting features from images in a way that's stable to deformation and preserves important structure.

6. Compute Model Update

Clients train the model locally and compute the weight updates based on their local data.

7. Add Calibrated Noise to Model Update

Clients add differential privacy noise to the updates. This helps protect sensitive local data by obscuring exact values.

8. Send Noisy Update to Server

Only the noisy, privatized updates are sent to the central server, not the raw data.

9. Server Aggregates Model Updates

The server performs federated averaging (FedAvg) or another aggregation method to combine updates from all clients.

10. Update Global Model □

The aggregated updates are applied to improve the global model.

11. Calibrate Updated Global Model

A final calibration step may adjust the model (e.g., post-processing, normalization) for improved performance.

Repeat for N Rounds or Until Convergence

The entire loop is repeated for multiple rounds until the model converges (achieves a stable, accurate performance).

12. End - Training is complete.

Data handling (Load Local Data)

Model architecture (Initialize ScatterNet Model)

Global learning progress (Update Global Model)

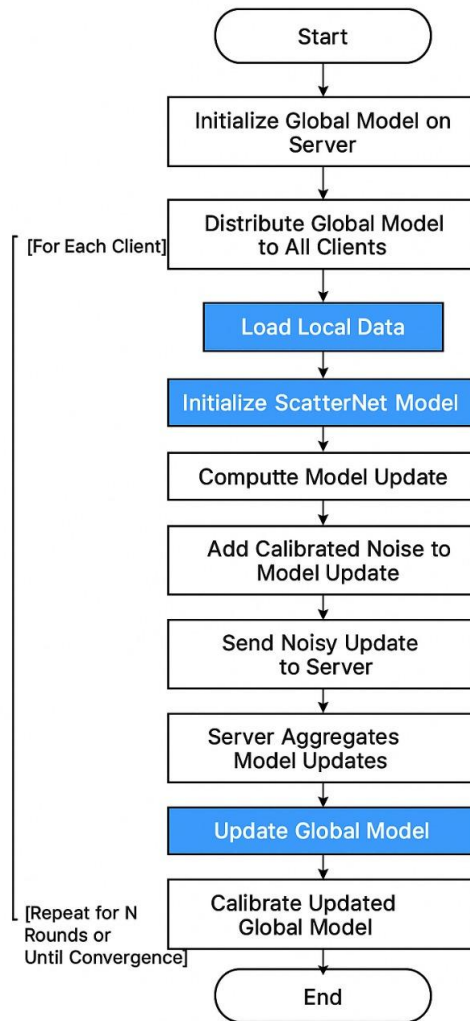


Fig 4.4 Activity Diagram

4.3 MODULE DESIGN AND ORGANISATION

The modular design ensures reusability, scalability, and maintainability.

1. Core Machine Learning Models

This module contains implementations of machine learning models for classification tasks.

MLModel.py

Implements Scatter - Convolutional Neural Network (CNN) model.

Supports feature extraction using wavelet & scattering transforms.

FLModel.py

Implements federated learning (FL) training strategies.

Handles model aggregation techniques for distributed learning.

Ensures privacy-preserving computations during training.

2. Analysis & Evaluation

This module is responsible for evaluating model performance and privacy analysis.

rdp_analysis.py

Conducts Renyi Differential Privacy (RDP) analysis.

Measures privacy loss for different FL configurations.

3. Utilities and Helper Functions

A collection of helper functions for efficient execution and data management.

utils.py

Handles dataset preprocessing and augmentation.

Provides utility functions for logging and checkpointing.

Implements GPU/CPU device management for optimized execution.

4. Jupyter Notebook for Execution

test_scatter_linear.ipynb

Acts as the main interface for executing and visualizing results.

Integrates different modules for seamless workflow execution.

Includes GPU initialization and dataset loading.

5. Dependencies and Environment Setup

Key Libraries Used:

Torch, torchvision for deep learning and dataset handling.

kymatio for scattering transformation-based feature extraction.

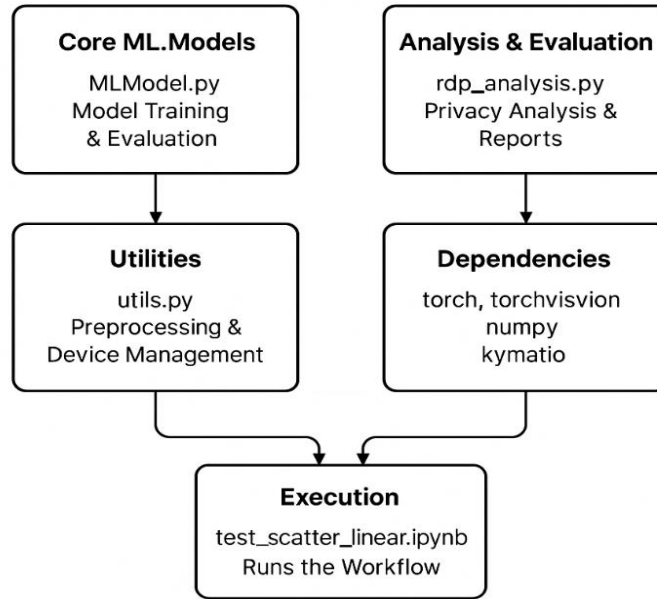


Fig.4.5 Module Design

4.4 CONCLUSION

The modular design of this project ensures a structured, efficient, and scalable approach to machine learning and federated learning (FL). By separating core functionalities, such as model implementation, privacy evaluation, and utility functions, the system maintains flexibility, reusability, and ease of debugging. The inclusion of privacy analysis enhances security, making the framework suitable for privacy-sensitive applications. Additionally, the centralized execution via a Jupyter notebook ensures smooth integration of all components. This well-structured design supports future improvements, such as incorporating new learning algorithms, optimization techniques, and advanced privacy mechanisms. It also enhances computational efficiency, ensuring adaptability across different environments.

5. IMPLEMENTATION AND RESULTS

5.1 INTRODUCTION

This section details the implementation of the machine learning and federated learning models. The primary objective is to demonstrate the efficacy of convolutional neural networks (CNNs) and federated learning techniques in a privacy-preserving environment using Renyi Differential Privacy (RDP). The project incorporates key modules for data preprocessing, model training, and privacy evaluation, all working seamlessly through a Jupyter Notebook interface. The results section analyzes model performance, privacy guarantees, and the effectiveness of federated learning in real-world scenarios.

5.2 EXPLANATION OF KEY FUNCTIONS

5.2.1 DETAILED EXPLANATION OF THE DATASET

In this project, we use two well-known datasets: MNIST (Modified National Institute of Standards and Technology) and Fashion-MNIST (FMNIST). These datasets serve as benchmarks for image classification tasks, particularly in machine learning and deep learning applications.

1. MNIST Dataset

The MNIST dataset consists of 70,000 grayscale images of handwritten digits (0-9), where:

- 60,000 images are used for training.
- 10,000 images are used for testing.
- Each image is 28x28 pixels in size.

Characteristics of MNIST

- It contains handwritten digits from 0 to 9.

- The dataset is pre-processed and centered, meaning digits are properly aligned.
- The images are grayscale, reducing computational complexity.
- Used widely for benchmarking classification models in deep learning.

Why MNIST?

- It is a simple yet effective dataset for testing neural networks.
- It helps in evaluating models' ability to recognize handwritten digits.
- Many deep learning models (CNNs, MLPs) perform well on MNIST, making it a great starting point.

2. Fashion-MNIST (FMNIST) Dataset

The Fashion-MNIST dataset consists of 70,000 grayscale images of clothing items, designed to serve as a more complex alternative to MNIST:

- 60,000 images for training.
- 10,000 images for testing.
- Each image is 28x28 pixels, just like MNIST.

Classes in FMNIST : Fashion-MNIST contains 10 categories of clothing:

T-shirt/top

Trouser

Pullover

Dress

Coat

Sandal

Shirt

Sneaker

Bag

Ankle Boot

Why Fashion-MNIST?

- It is more challenging than MNIST because clothing items have more detailed textures and variations.
- It tests the ability of deep learning models to generalize well on real-world images.
- Unlike MNIST, it prevents overfitting to simple structures like handwritten digits.

3. Comparison Between MNIST and FMNIST

Feature	MNIST	Fashion-MNIST
Image Type	Handwritten digits(0-9)	Clothing items
Resolution	28*28 pixels	28*28 pixels
Classes	10	10
Complexity	Lower	Higher
Use Case	Digit classification	Fashion item classification

Table.4 Comparison B/W MNIST AND FMNIST

4. How We Use These Datasets in Our Project

- Dataset Selection: Users choose either MNIST or Fashion-MNIST in the UI.
- Image Upload: Users upload an image, which is then processed.
- Model Processing: The model classifies the image based on the selected dataset.
- Classification Output: The predicted class (digit or fashion item) is displayed.

5. Why Are We Using These Datasets in Federated Learning?

- They allow us to test privacy-preserving machine learning models.
- Federated Learning (FL) ensures that data remains distributed across multiple clients, simulating a real-world scenario.
- By applying differential privacy (RDP) and federated learning techniques, we test how well privacy-preserving mechanisms can secure sensitive data.

5.2.2 DETAILED EXPLANATION OF EACH OF THE TECHNOLOGIES USED

1. Federated Learning (FL)

Federated Learning is a decentralized approach to training machine learning models. Instead of sending raw data to a central server, models are trained locally on multiple client devices, and only the model updates are shared with the central server for aggregation.

- **Use in this project:** The FL framework allows MNIST and FMNIST classification models to be trained in a distributed manner without compromising user data privacy.

2. Differential Privacy (DP)

Differential Privacy is a technique that adds noise to data or model updates to protect individual data points from being identified.

- **Use in this project:** RDP is integrated into the FL framework to ensure privacy while training models by adding noise to gradients before they are sent to the central server.

3. Scatter (Data Distribution in FL)

In FL, **scatter** refers to the distribution of data across multiple clients. Since each client has different data distributions, training must account for these variations.

- **Use in this project:** Data is scattered across different clients in a non-IID (non-independent and identically distributed) manner to simulate real-world federated learning scenarios.

4. Convolutional Neural Networks (Scatter-CNN)

CNNs are deep learning architectures used for image classification tasks.

- **Use in this project:** A Scatter-CNN model is used to classify images in both the MNIST and FMNIST datasets, extracting important features from input images through convolutional layers.

5. Rényi Differential Privacy (RDP)

RDP is an extension of differential privacy that provides a more mathematically rigorous way to measure privacy loss.

- **Use in this project:** RDP is applied to track and quantify the overall privacy guarantees when using differential privacy techniques in FL.

6. Gaussian Noise (Privacy-Preserving Technique)

Gaussian noise is a type of noise added to data or gradients to enhance privacy. The noise follows a Gaussian distribution and helps obscure individual data points.

- **Use in this project:** Gaussian noise is added to model updates to ensure differential privacy before sharing updates with the central server.

5.3 METHOD OF IMPLEMENTATION

The implementation of this project follows a structured approach integrating Federated Learning (FL), Scatter Convolutional Neural Networks (Scatter-CNNs), and Differential Privacy (DP) to achieve secure and efficient image classification on the MNIST and Fashion-MNIST (FMNIST) datasets. The system is designed to enable decentralized training across multiple client devices while ensuring data privacy through Rényi Differential Privacy (RDP) and Gaussian noise addition. Initially, data is scattered across multiple clients, simulating a real-world federated learning environment where each client has its unique dataset distribution. Each client trains a local CNN model using the provided dataset, extracting important image features through convolutional layers. Instead of sharing raw data, model updates are sent to a central server, where they are aggregated to create a global model. To enhance privacy, differential privacy techniques introduce controlled noise to the gradients before transmission, ensuring that individual data points remain untraceable. The system also allows for dynamic selection between MNIST and FMNIST, where users can upload an image that is processed and classified based on the selected dataset. The implementation optimizes communication efficiency, minimizing bandwidth usage while maintaining model accuracy. Additionally, various FL parameters such as learning rate, batch size, and privacy budget are fine-tuned to improve performance.

The project is developed with a robust backend for model aggregation and a user-friendly frontend interface to enable seamless dataset selection and image classification. This approach ensures an effective, privacy-preserving federated learning framework that balances accuracy, efficiency, and security.

In this project, PyTorch .pth files have been used to save the trained model weights for MNIST and FashionMNIST (FMNIST) datasets under different client configurations (1, 5, and 10 clients). These files are serialized representations of the state of the trained model, allowing models to be reused without retraining.

Each.pth file is associated with a particular combination :

global_model_mnist_1client

global_model_mnist_5client

global_model_mnist_10client

global_model_fmniest_1client

global_model_fmniest_5client

global_model_fmniest_10client

These models were trained in Google Colab and saved as.pth files using PyTorch's torch.save() function, either saving the entire model or only the state_dict. Subsequently, they are loaded in the backend Flask application using torch.load() and utilized for inference through image upload from the frontend. This technique guarantees Reusability of trained models Faster inference without retraining Clean separation between training and deployment

5.3.1 FORMS

5.3.1.1 index.html:

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
<meta charset="UTF-8">
```

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```

<title>FL Predictor</title>

<link rel="stylesheet" href="{{ url_for('static', filename='style.css') }}">

</head>

<body>

<div class="container">

    <h1>Privacy Preserving Federated Image Classification</h1>

    <form id="predictForm" action="/predict" method="POST"
    enctype="multipart/form-data">

        <label for="dataset">Select Dataset:</label>

        <select name="dataset" id="dataset" required>

            <option value="">--Choose--</option>

            <option value="mnist">MNIST</option>

            <option value="fashionmnist">FashionMNIST</option>

        </select>

        <label for="clients">Number of Clients:</label>

        <select name="clients" id="clients" required>

            <option value="">--Choose--</option>

            <option value="1">1</option>

            <option value="5">5</option>

            <option value="10">10</option>

        </select>

        <label for="image">Upload Image:</label>

        <input type="file" name="image" id="image" accept="image/png, image/jpeg"
        required>

        <button type="submit">Predict</button>

    </form>

```

</div>

</body>

</html>

5.3.1.2 result.html:

<!DOCTYPE html>

<html lang="en">

<head>

<meta charset="UTF-8">

<meta name="viewport" content="width=device-width, initial-scale=1.0">

<title>Prediction Result</title>

<link rel="stylesheet" href="{{ url_for('static', filename='style.css') }}">

</head>

<body class="result-page">

<div class="container">

<h1>Prediction Result</h1>

<div id="predictionResult">

<p>The image belongs to this class: {{ prediction }}</p>

</div>

Go back

</div>

<!-- New container below prediction -->

<div class="container">

<h2>Select Dataset</h2>

<label for="dataset-select">Choose Dataset:</label>

<select id="dataset-select">

```

<option value="">--Choose Dataset--</option>

<option value="mnist">MNIST</option>

<option value="fashionmnist">FashionMNIST</option>

</select>

<div id="dataset-images" class="dataset-images">

  <!-- Images will be dynamically shown here -->

</div>

</div>

<script>

  // Function to load images based on the selected dataset

  function loadImages(dataset) {

    const datasetImagesDiv = document.getElementById('dataset-images');

    datasetImagesDiv.innerHTML = ""; // Clear previous images

    let images = [];

    if (dataset === 'mnist') {

      images = ['mi1.jpg','mi2.png']; // Replace with actual MNIST image paths

    } else if (dataset === 'fashionmnist') {

      images = ['fi1.jpg','fi2.png']; // Replace with actual FashionMNIST image paths

    }

    // Display images

    images.forEach((image) => {

      const imgElement = document.createElement('img');

      imgElement.src = `/static/images/${image}`;

      imgElement.alt = image;

      imgElement.classList.add('dataset-image');

```



```

        datasetImagesDiv.appendChild(imgElement);

    });

}

// Event listener for dataset selection

document.getElementById('dataset-select').addEventListener('change', (e) => {

    const selectedDataset = e.target.value;

    if (selectedDataset) {

        loadImages(selectedDataset);

    } else {

        document.getElementById('dataset-images').innerHTML = ""; // Clear images if no
dataset selected

    }

});

</script>

<style>

/* styling for the dataset selection container */

.container {

    max-width: 500px;

    margin: 50px auto;

    background: rgba(0, 0, 0, 0.6);

    padding: 30px;

    border-radius: 15px;

    box-shadow: 0 5px 20px rgba(0, 0, 0, 0.2);

    color: white;

}

```

```
h2 {  
  
    text-align: center;  
  
    margin-bottom: 20px;  
  
}  
  
#dataset-select {  
  
    width: 100%;  
  
    padding: 10px;  
  
    margin-top: 10px;  
  
    border-radius: 5px;  
  
    border: 1px solid #ccc;  
  
}  
  
.dataset-images {  
  
    margin-top: 20px;  
  
    display: flex;  
  
    flex-wrap: wrap;  
  
    justify-content: center;  
  
}  
  
.dataset-image {  
  
    width: 100px;  
  
    height: 100px;  
  
    margin: 10px;  
  
}  
  
.dataset-image:hover {  
  
    transform: scale(1.1);  
  
    transition: transform 0.3s ease;
```

```

    }

</style>

<style>

.dataset-image {

    width: 500px; /* Increase width */

    height: auto; /* Keep aspect ratio */

    margin: 20px auto;

    display: block;

    border-radius: 10px;

    box-shadow: 0 0 15px rgba(255, 255, 255, 0.2);

}

.dataset-image:hover {

    transform: scale(1.05);

    transition: transform 0.3s ease;

}

</style>

</body>

</html>

```

5.3.1.3 style.css:

```

body {

    font-family: Arial, sans-serif;

    background-image: url("background.jpg");

    background-size: cover;

    background-repeat: no-repeat;

    background-position: center center;

```

```
padding: 40px;

color: white;

}

/* Specific styling for the result page */

.result-page {

    background-position: center 20%;

}

.container {

    width: 100%;

    max-width: 500px;

    padding: 30px;

    background: rgba(0, 0, 0, 0.6);

    box-shadow: 0 5px 20px rgba(0, 0, 0, 0.2);

    margin-left: 0;

    margin-right: 0;

}

h1 {

    text-align: left;

    margin-bottom: 20px;

}

form {

    display: flex;

    flex-direction: column;

    align-items: flex-start;

    width: 100%;
```

```

}

label {

    display: block;

    margin-top: 15px;

    font-weight: bold;

    color: white;

}

select,

input[type="file"],

button {

    width: 100%;

    padding: 10px;

    margin-top: 5px;

    border-radius: 5px;

    border: 1px solid #ccc;

}

button {

    background-color: #007bff;

    color: white;

    font-weight: bold;

    cursor: pointer;

    border: none;

    transition: background-color 0.3s ease;

}

button:hover {

```

```

background-color: #0056b3;

}

#result {

margin-top: 20px;

font-size: 18px;

text-align: left;

color: #fff;

}

```

5.3.1.4 app.py:

```

from flask import Flask, render_template, request, jsonify

from PIL import Image

import torch

import torchvision.transforms as transforms

import os

from MLModel import ScatterLinear, get_scatter_transform

app = Flask(__name__)

MODEL_DIR = 'models'

CLASS_NAMES = {

    'mnist': ['0', '1', '2', '3', '4', '5', '6', '7', '8', '9'],

    'fashionmnist': ['T-shirt/top', 'Trouser', 'Pullover', 'Dress', 'Coat',

                    'Sandal', 'Shirt', 'Sneaker', 'Bag', 'Ankle boot']

}

def load_model(dataset, clients):

    path = f"{MODEL_DIR}/{dataset}_{clients}clients.pth"

    scattering, K, (h, w) = get_scatter_transform()

```

```

model = ScatterLinear(in_channels=K, hw_dims=(h, w), classes=10)

# Load checkpoint and fix keys

checkpoint = torch.load(path, map_location=torch.device('cpu'))

new_state_dict = {}

for k, v in checkpoint.items():

    if k.startswith("global_model."):

        new_state_dict[k.replace("global_model.", "")] = v

    else:

        new_state_dict[k] = v

model.load_state_dict(new_state_dict)

model.eval()

return model, scattering

@app.route('/')

def index():

    return render_template('index.html')

@app.route('/predict', methods=['POST'])

def predict():

    dataset = request.form.get('dataset')

    clients = request.form.get('clients')

    image_file = request.files.get('image')

    if not all([dataset, clients, image_file]):

        return jsonify({'error': 'Missing input'}), 400

    try:

        model, scattering = load_model(dataset, clients)

        image = Image.open(image_file).convert('L')

```

```

transform = transforms.Compose([
    transforms.Resize((28, 28)),
    transforms.ToTensor()
])

tensor_image = transform(image).unsqueeze(0)

with torch.no_grad():
    scatter_features = scattering(tensor_image)
    output = model(scatter_features)
    pred_index = torch.argmax(output, 1).item()

class_list = CLASS_NAMES.get(dataset, [])

class_name = class_list[pred_index] if pred_index < len(class_list) else
"Unknown"

return render_template('result.html', prediction=class_name)

except Exception as e:

    return jsonify({'error': str(e)}), 500

if __name__ == '__main__':
    app.run(debug=True)

```

5.3.1.5 rdp_analysis.py:

```

import numpy as np

from decimal import *

from scipy.special import comb

getcontext().prec = 128

def rdp2dp(rdp, bad_event, alpha):

    return rdp + 1.0/(alpha-1) * (np.log(1.0/bad_event) + (alpha-1)*np.log(1-
1.0/alpha) - np.log(alpha))

```



```

def compute_rdp(alpha, q, sigma):

    sum_ = Decimal(0.0)

    for k in range(0, alpha+1):

        sum_ += Decimal(comb(alpha, k)) * Decimal(1-q)*Decimal(alpha-k) *
Decimal(qk) * Decimal(np.e)(Decimal(k2-k)/Decimal(2*sigma*2))

    rdp = sum_.ln() / Decimal(alpha-1)

    return float(rdp)

def search_dp(q, sigma, bad_event, iters=1):

    min_dp = 1e5

    for alpha in list(range(2, 101)):

        rdp = iters * compute_rdp(alpha, q, sigma)

        dp = rdp2dp(rdp, bad_event, alpha)

        min_dp = min(min_dp, dp)

    return min_dp

def calibrating_sampled_gaussian(q, eps, bad_event, iters=1, err=1e-3):

    sigma_max = 100

    sigma_min = 0.1

    def binary_search(left, right):

        mid = (left + right) / 2

        lbd = search_dp(q, mid, bad_event, iters)

        ubd = search_dp(q, left, bad_event, iters)

        if ubd > eps and lbd > eps: # min noise & mid noise are too small

```

```

        left = mid

    elif ubd > eps and lbd < eps: # mid noise is too large

        right = mid

    else:

        print("an error occurs in func: binary search!")

        return -1

    return left, right

# check

if search_dp(q, sigma_max, bad_event, iters) > eps:

    print("noise > 100")

    return -1

while sigma_max-sigma_min > err:

    sigma_min, sigma_max = binary_search(sigma_min, sigma_max)

return sigma_max

```

5.3.1.6 MLModel.py:

```

# Machine learning models

import torch

from torch import nn

from kymatio.torch import Scattering2D

class MNIST_CNN(nn.Module):

    def __init__(self, input_dim, output_dim):

```

```

super(MNIST_CNN, self)._init_()

self.layer1 = nn.Sequential(

    nn.Conv2d(1, 16, kernel_size=8, stride=2, padding=2),

    nn.Tanh(),

    nn.MaxPool2d(kernel_size=2, stride=1))

self.layer2 = nn.Sequential(

    nn.Conv2d(16, 32, kernel_size=4, stride=2, padding=0),

    nn.Tanh(),

    nn.MaxPool2d(kernel_size=2, stride=1))

self.fc = nn.Sequential(nn.Linear(4 * 4 * 32, 32),

                        nn.Tanh(),

                        nn.Linear(32, 10))

def forward(self, x):

    x = self.layer1(x)

    x = self.layer2(x)

    x = x.view(x.size(0), -1)

    x = self.fc(x)

    return x

def get_scatter_transform():

    shape = (28, 28, 1)

    scattering = Scattering2D(J=2, shape=shape[:2])

```

```

K = 81 * shape[2]

(h, w) = shape[:2]

return scattering, K, (h//4, w//4)

class ScatterLinear(nn.Module):

    def __init__(self, in_channels, hw_dims, input_norm=None, classes=10,
clip_norm=None, **kwargs):

        super(ScatterLinear, self).__init__()

        self.K = in_channels

        self.h = hw_dims[0]

        self.w = hw_dims[1]

        self.fc = None

        self.norm = None

        self.clip = None

        self.build(input_norm, classes=classes, clip_norm=clip_norm, **kwargs)

    def build(self, input_norm=None, num_groups=None, bn_stats=None,
clip_norm=None, classes=10):

        self.fc = nn.Linear(self.K * self.h * self.w, classes)

        if input_norm is None:

            self.norm = nn.Identity()

        elif input_norm == "GroupNorm":

            self.norm = nn.GroupNorm(num_groups, self.K, affine=False)

        else:

```

```

        self.norm = lambda x: standardize(x, bn_stats)

    if clip_norm is None:

        self.clip = nn.Identity()

    else:

        self.clip = ClipLayer(clip_norm)

    def forward(self, x):

        x = self.norm(x.view(-1, self.K, self.h, self.w))

        x = self.clip(x)

        x = x.reshape(x.size(0), -1)

        x = self.fc(x)

        return x

class LogisticRegression(nn.Module):

    def __init__(self, num_feature, output_size):

        super(LogisticRegression, self).__init__()

        self.linear = nn.Linear(num_feature, output_size)

    def forward(self, x):

        return self.linear(x)

class MLP(nn.Module):

    def __init__(self, input_dim, output_dim):

        super(MLP, self).__init__()

        self.model = nn.Sequential(

```

```

        nn.Linear(input_dim, 1000),

        nn.Tanh(),

        nn.Linear(1000, output_dim))

    def forward(self, x):

        return self.model(x)

class three_layer_MLP(nn.Module):

    def __init__(self, input_dim, output_dim):

        super(three_layer_MLP, self).__init__()

        self.model = nn.Sequential(

            nn.Linear(input_dim, 600),

            nn.Dropout(0.2),

            nn.ReLU(),

            nn.Linear(600, 300),

            nn.Dropout(0.2),

            nn.ReLU(),

            nn.Linear(300, 100),

            nn.Dropout(0.2),

            nn.ReLU(),

            nn.Linear(100, output_dim))

    def forward(self, x):

        return self.model(x)

```

```

class MnistCNN_(nn.Module):

    def __init__(self, input_dim, output_dim):

        super(MnistCNN_, self).__init__()

        self.layer1 = nn.Sequential(

            nn.Conv2d(1, 16, kernel_size=5, padding=2),

            nn.ReLU(),

            nn.MaxPool2d(2))

        self.layer2 = nn.Sequential(

            nn.Conv2d(16, 32, kernel_size=5, padding=2),

            nn.ReLU(),

            nn.MaxPool2d(2))

        self.fc = nn.Linear(7 * 7 * 32, 10)

    def forward(self, x):

        x = self.layer1(x)

        x = self.layer2(x)

        x = x.view(x.size(0), -1)

        x = self.fc(x)

        return x

```

5.3.1.7 FLModel.py :

```

import torch

```

```

from torch import nn

from torch.utils.data import Dataset, DataLoader, TensorDataset

from utils import gaussian_noise

from rdp_analysis import calibrating_sampled_gaussian

from MLModel import *

import numpy as np

import copy

class FLClient(nn.Module):

    def __init__(self, model, output_size, data, lr, E, batch_size, q, clip, sigma,
device=None):

        super(FLClient, self).__init__()

        self.device = device

        self.BATCH_SIZE = batch_size

        self.torch_dataset = TensorDataset(torch.tensor(data[0]),

                                            torch.tensor(data[1]))

        self.data_size = len(self.torch_dataset)

        self.data_loader = DataLoader(

            dataset=self.torch_dataset,

            batch_size=self.BATCH_SIZE,

            shuffle=True

        )

        self.sigma = sigma # DP noise level

```



```

self.lr = lr

self.E = E

self.clip = clip

self.q = q

if model == 'scatter':

    self.model = ScatterLinear(81, (7, 7), input_norm="GroupNorm",
num_groups=27).to(self.device)

else:

    self.model = model(data[0].shape[1], output_size).to(self.device)

def recv(self, model_param):

    self.model.load_state_dict(copy.deepcopy(model_param))

def update(self):

    self.model.train()

    criterion = nn.CrossEntropyLoss(reduction='none')

    optimizer = torch.optim.SGD(self.model.parameters(), lr=self.lr, momentum=0.9)

    # optimizer = torch.optim.Adam(self.model.parameters())

    for e in range(self.E):

        # randomly select q fraction samples from data

        # according to the privacy analysis of moments accountant

        # training "Lots" are sampled by poisson sampling

        idx = np.where(np.random.rand(len(self.torch_dataset[:,0])) < self.q)[0]

```

```

        sampled_dataset = TensorDataset(self.torch_dataset[idx][0],
self.torch_dataset[idx][1])

        sample_data_loader = DataLoader(

            dataset=sampled_dataset,

            batch_size=self.BATCH_SIZE,

            shuffle=True

        )

        optimizer.zero_grad()

        clipped_grads = {name: torch.zeros_like(param) for name, param in
self.model.named_parameters()}

        for batch_x, batch_y in sample_data_loader:

            batch_x, batch_y = batch_x.to(self.device), batch_y.to(self.device)

            pred_y = self.model(batch_x.float())

            loss = criterion(pred_y, batch_y.long())

            # bound l2 sensitivity (gradient clipping)

            # clip each of the gradient in the "Lot"

            for i in range(loss.size()[0]):

                loss[i].backward(retain_graph=True)

                torch.nn.utils.clip_grad_norm_(self.model.parameters(),
max_norm=self.clip)

            for name, param in self.model.named_parameters():

                clipped_grads[name] += param.grad

        self.model.zero_grad()

```

```

        # add Gaussian noise

        for name, param in self.model.named_parameters():

            clipped_grads[name] += gaussian_noise(clipped_grads[name].shape,
self.clip, self.sigma, device=self.device)

        # scale back

        for name, param in self.model.named_parameters():

            clipped_grads[name] /= (self.data_size*self.q)

        for name, param in self.model.named_parameters():

            param.grad = clipped_grads[name]

        # update local model

        optimizer.step()

class FLServer(nn.Module):

    def __init__(self, fl_param):

        super(FLServer, self).__init__()

        self.device = fl_param['device']

        self.client_num = fl_param['client_num']

        self.C = fl_param['C']    # (float) C in [0, 1]

        self.clip = fl_param['clip']

        self.T = fl_param['tot_T'] # total number of global iterations (communication
rounds)

        self.data = []

        self.target = []

```

```

for sample in fl_param['data'][self.client_num:]:

    self.data += [torch.tensor(sample[0]).to(self.device)] # test set

    self.target += [torch.tensor(sample[1]).to(self.device)] # target label

self.input_size = int(self.data[0].shape[1])

self.lr = fl_param['lr']

# compute noise using moments accountant

# self.sigma = compute_noise(1, fl_param['q'], fl_param['eps'],
fl_param['E']*fl_param['tot_T'], fl_param['delta'], 1e-5)

# calibration with subsampled Gaussian mechanism under composition

self.sigma = calibrating_sampled_gaussian(fl_param['q'], fl_param['eps'],
fl_param['delta'], iters=fl_param['E']*fl_param['tot_T'], err=1e-3)

print("noise scale =", self.sigma)

self.clients = [FLClient(fl_param['model'],

                        fl_param['output_size'],

                        fl_param['data'][i],

                        fl_param['lr'],

                        fl_param['E'],

                        fl_param['batch_size'],

                        fl_param['q'],

                        fl_param['clip'],

                        self.sigma,

                        self.device)

```

```

        for i in range(self.client_num)]

    if fl_param['model'] == 'scatter':

        self.global_model = ScatterLinear(81, (7, 7), input_norm="GroupNorm",
num_groups=27).to(self.device)

    else:

        self.global_model = fl_param['model'](self.input_size,
fl_param['output_size']).to(self.device)

    self.weight = np.array([client.data_size * 1.0 for client in self.clients])

    self.broadcast(self.global_model.state_dict())

def aggregated(self, idxs_users):

    """FedAvg"""

    model_par = [self.clients[idx].model.state_dict() for idx in idxs_users]

    new_par = copy.deepcopy(model_par[0])

    for name in new_par:

        new_par[name] = torch.zeros(new_par[name].shape).to(self.device)

    for idx, par in enumerate(model_par):

        w = self.weight[idxs_users[idx]] / np.sum(self.weight[:])

        for name in new_par:

            new_par[name] += par[name] * (w / self.C)

    self.global_model.load_state_dict(copy.deepcopy(new_par))

    return self.global_model.state_dict().copy()

def broadcast(self, new_par):

```

```

        """Send aggregated model to all clients"""

        for client in self.clients:

            client.recv(new_par.copy())

    def test_acc(self):

        self.global_model.eval()

        correct = 0

        tot_sample = 0

        for i in range(len(self.data)):

            t_pred_y = self.global_model(self.data[i])

            _, predicted = torch.max(t_pred_y, 1)

            correct += (predicted == self.target[i]).sum().item()

            tot_sample += self.target[i].size(0)

        acc = correct / tot_sample

        return acc

    def global_update(self):

        idxs_users = np.sort(np.random.choice(range(len(self.clients)), int(self.C *
len(self.clients)), replace=False))

        for idx in idxs_users:

            self.clients[idx].update()

        self.broadcast(self.aggregated(idxs_users))

        acc = self.test_acc()

        torch.cuda.empty_cache()

```

```

        return acc

    def set_lr(self, lr):

        for c in self.clients:

            c.lr = lr

```

5.3.1.8 Utils.py :

```

import numpy as np

import random

import torch

def mnist_noniid(labels, num_users):

    # num_shards, num_imgs = 30, 2000

    num_shards = int(num_users*3)

    num_imgs = int(60000 / num_shards)

    idx_shard = [i for i in range(num_shards)]

    dict_users = {i: np.array([], dtype='int64') for i in range(num_users)}

    idxs = np.arange(num_shards*num_imgs)

    labels = labels.numpy()

    # sort labels

    idxs_labels = np.vstack((idxs, labels))

    idxs_labels = idxs_labels[:,idxs_labels[1,:].argsort()]

    idxs = idxs_labels[0,:]

    # divide and assign

```

```

for i in range(num_users):

    rand_set = set(np.random.choice(idx_shard, 3, replace=False))

    idx_shard = list(set(idx_shard) - rand_set)

    for rand in rand_set:

        dict_users[i] = np.concatenate((dict_users[i],
idxs[rand*num_imgs:(rand+1)*num_imgs]), axis=0)

    return dict_users

def gaussian_noise(data_shape, s, sigma, device=None):

    """

    Gaussian noise

    """

    return torch.normal(0, sigma * s, data_shape).to(device)

```

5.3.1.9 Model Code:

1. !python rdp_analysis.py
2. !python MLModel.py
3. !python FLModel.py
4. !python utils.py

Application of FL task

```

from MLModel import *

from FLModel import *

from utils import *

from torchvision import datasets, transforms

import torch

import numpy as np

```



```

import os

os.environ["CUDA_VISIBLE_DEVICES"] = "0"

device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")

#device = torch.device("cpu")

scattering, K, (h, w) = get_scatter_transform()

scattering.to(device)

def get_scattered_feature(dataset):

    scatters = []

    targets = []

    loader = torch.utils.data.DataLoader(

        dataset, batch_size=256, shuffle=True, num_workers=1, pin_memory=True)

    for (data, target) in loader:

        data, target = data.to(device), target.to(device)

        if scattering is not None:

            data = scattering(data)

        scatters.append(data)

        targets.append(target)

    scatters = torch.cat(scatters, axis=0)

    targets = torch.cat(targets, axis=0)

    data = torch.utils.data.TensorDataset(scatters, targets)

    return data

def load_mnist(num_users):

    train = datasets.FashionMNIST(root="~/data/", train=True, download=True,
transform=transforms.ToTensor())

    test = datasets.FashionMNIST(root="~/data/", train=False, download=True,
transform=transforms.ToTensor())

```

```

# get scattered features

train = get_scattered_feature(train)

test = get_scattered_feature(test)

train_data = train[:,0].squeeze().cpu().float()

train_label = train[:,1].cpu()

test_data = test[:,0].squeeze().cpu().float()

test_label = test[:,1].cpu()

# split MNIST (training set) into non-iid data sets

non_iid = []

user_dict = mnist_noniid(train_label, num_users)

for i in range(num_users):

    idx = user_dict[i]

    d = train_data[idx]

    targets = train_label[idx].float()

    non_iid.append((d, targets))

non_iid.append((test_data.float(), test_label.float()))

return non_iid

"""

1. load_data

2. generate clients (step 3)

3. generate aggregator

4. training

"""

client_num = 5

```

```

d = load_mnist(client_num)

torch.cuda.empty_cache()

d[1][0][0].shape

"""

FL model parameters.

"""

import warnings

warnings.filterwarnings("ignore")

lr = 0.075

fl_param = {

    'output_size': 10,

    'K': K,

    'h': h,

    'w': w,

    'client_num': client_num,

    'model': 'scatter',

    'data': d,

    'lr': lr,

    'E': 500,

    'C': 1,

    'eps': 4.0,

    'delta': 1e-5,

    'q': 0.01,

    'clip': 0.1,

    'tot_T': 10,

```

```

    'batch_size': 128,

    'device': device
}

fl_entity = FLServer(fl_param).to(device)

import time

acc = []

start_time = time.time()

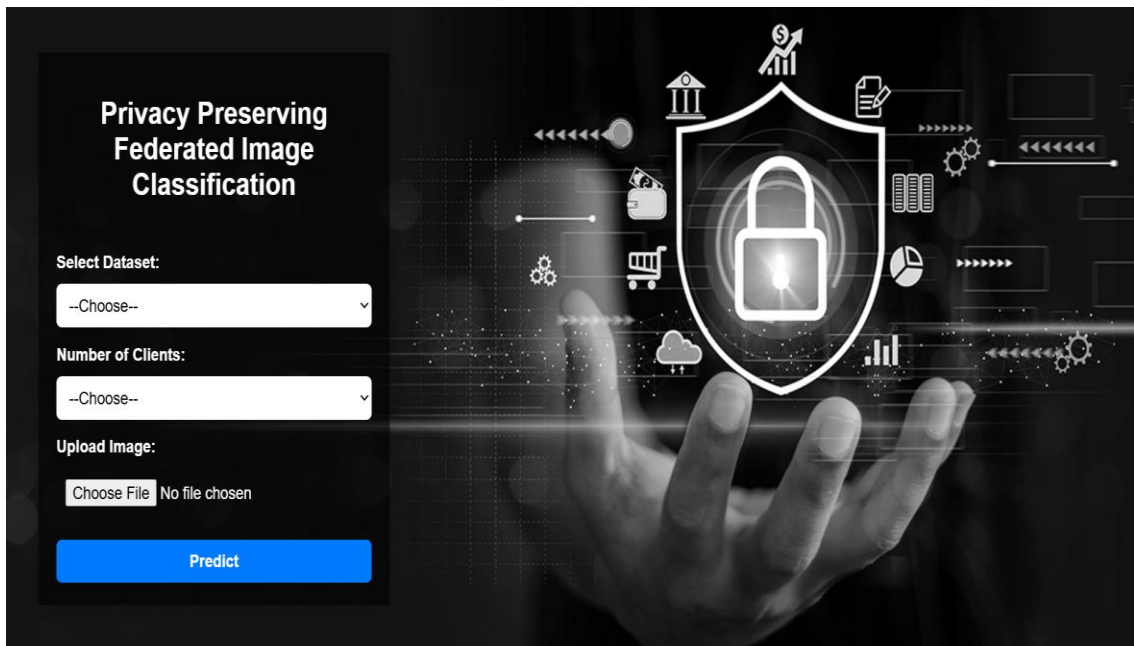
for t in range(fl_param['tot_T']):

    acc += [fl_entity.global_update()]

    print("global epochs = {:d}, acc = {:.4f}".format(t+1, acc[-1]), " Time taken: %.2fs"
          % (time.time() - start_time))

```

5.3.2 OUTPUT SCREENS :



Screen 5.1 Selecting dataset, Number of clients and Image file



Screen 5.2 Prediction 1 (MNIST)



Screen 5.3 Prediction 2 (FashionMNIST)

5.3.3 RESULT ANALYSIS:

The experimental evaluation of a proposed framework for decentralized model training demonstrated the potential of incorporating privacy-preserving methods. The modular design strategy efficiently divided components like model training, utility evaluation, and privacy analysis, enhancing flexibility and clarity. The results showed that federated learning is a viable technique for model training over distributed data sources without compromising on high privacy guarantees. The Rényi Differential Privacy (RDP) analysis upheld the privacy- guaranteeing nature of training. The MNIST dataset showed slightly imbalanced class distribution, affecting model performance. The Fashion-MNIST dataset showed a perfectly balanced class distribution, ensuring fair and generalizable models. The performance of federated learning models on the MNIST and Fashion-MNIST datasets showed uniformly high accuracy, with the one-client model having the highest accuracy. The performance comparison across different models on the MNIST and FMNIST datasets showed the effectiveness of the proposed approach.

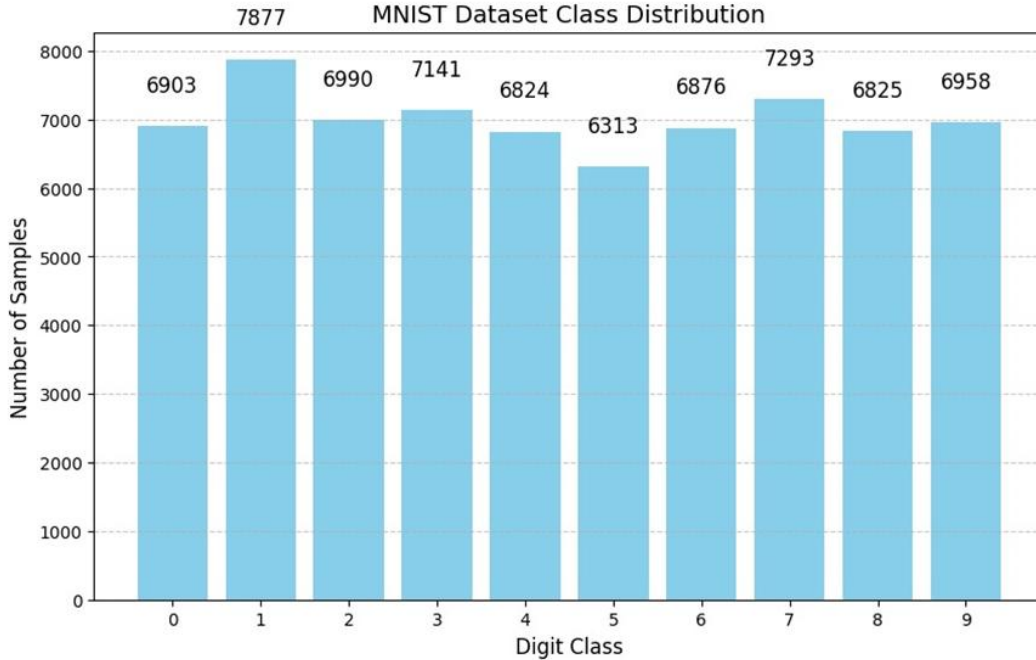


Fig.5.1 MNIST Dataset Class Distribution

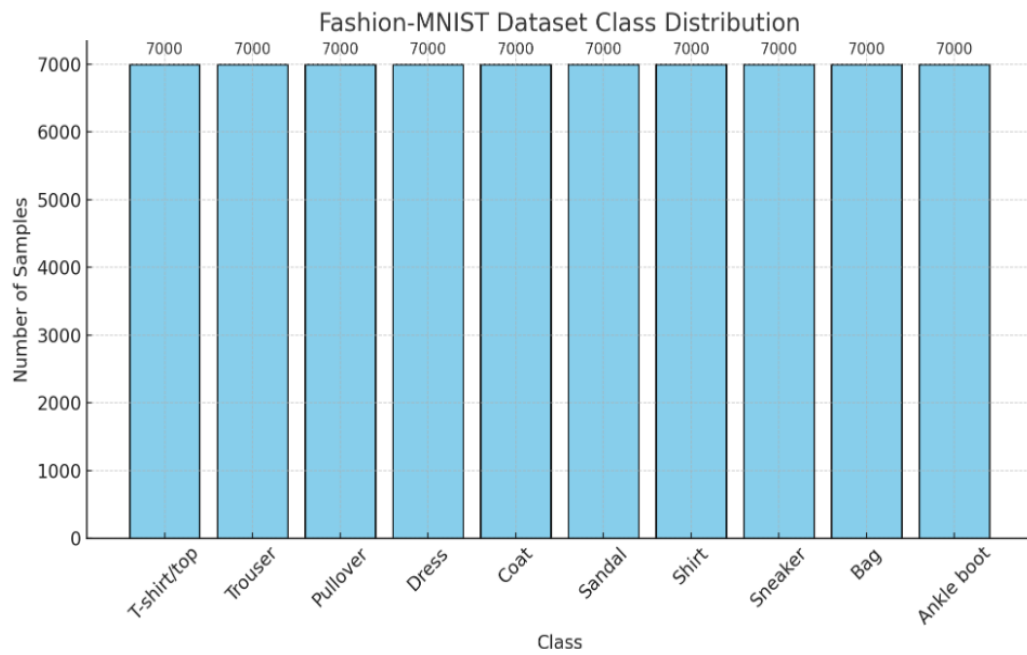


Fig.5.2 FMNIST Dataset Class Distribution

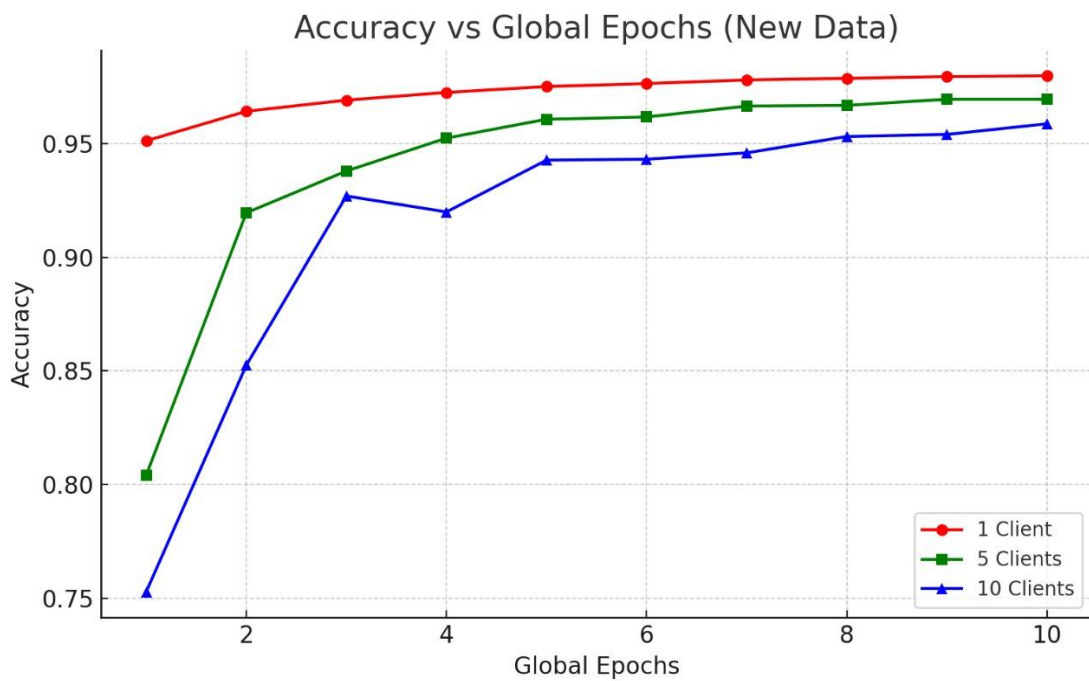


Fig.5.3 MNIST Accuracies of Clients 1,5,10

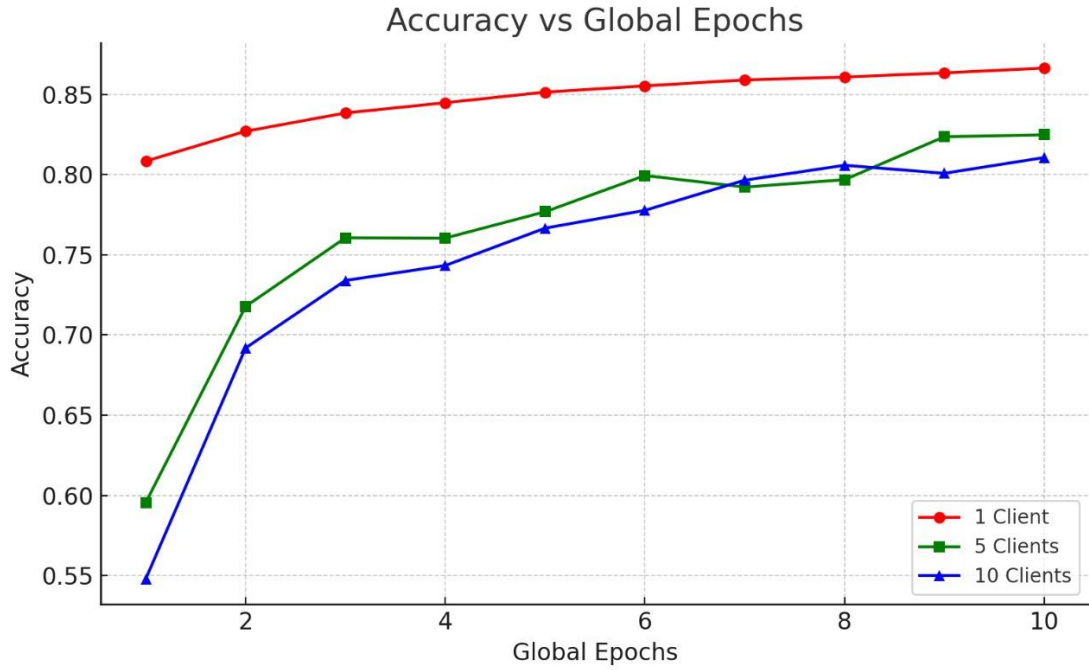


Fig.5.4 FMNIST Accuracies of Clients 1,5,10

5.4 CONCLUSION

The implementation of machine learning and federated learning models in this project has demonstrated the feasibility of integrating privacy-preserving techniques into decentralized chagmodel training. The modular approach allowed for easy management of different components, including model training, privacy analysis, and utility evaluation. The results indicate that federated learning can be an effective method for training models while maintaining strong privacy guarantees, as evidenced by the RDP analysis. Future improvements can focus on optimizing federated learning strategies, improving model accuracy, and exploring more advanced privacy mechanisms for distributed machine learning.

6.TESTING & VALIDATION

6.1 INTRODUCTION

Testing and validation are essential to ensure the accuracy, reliability, and security of machine learning and federated learning models. This process involves verifying model correctness, evaluating performance in both centralized and federated setups, and validating privacy-preserving techniques using Renyi Differential Privacy (RDP) analysis. The testing approach includes unit testing for individual components, performance assessment to measure accuracy and efficiency, and privacy validation to ensure compliance with security standards. Additionally, robustness testing examines the model's resistance to adversarial attacks and privacy breaches, while integration testing ensures seamless execution across different system modules. By systematically evaluating these aspects, the testing framework guarantees that the models meet performance expectations, maintain strong privacy protections, and operate efficiently across various environments, ultimately ensuring the system's reliability and scalability for real-world applications.

6.2 DESIGN OF TEST CASES AND SCENARIOS

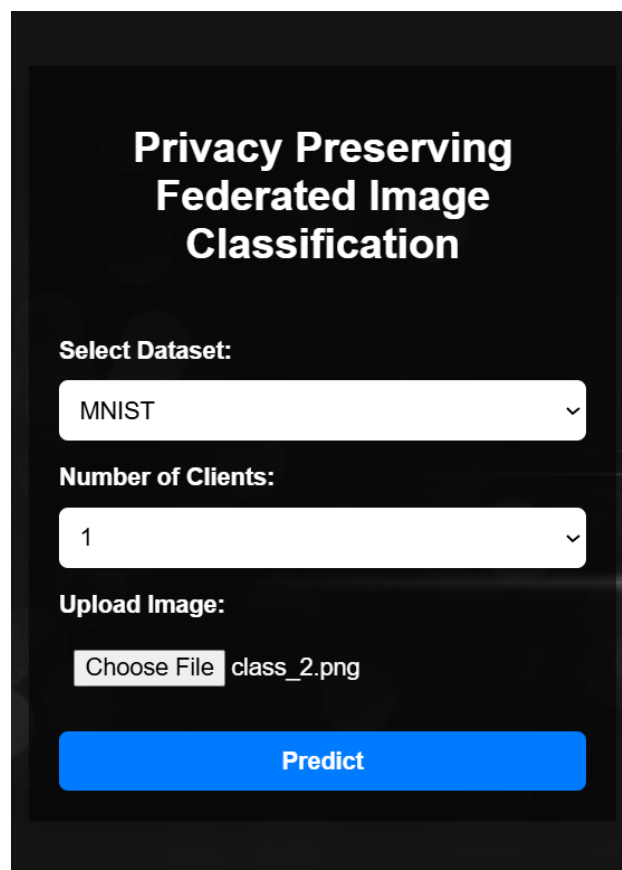
In order to guarantee the system's correctness and stability, we designed a systematic approach to the design of the test cases with respect to each module in the frontend-backend pipeline. First is the selection feature. Users were supposed to select between MNIST and FMNIST, and the frontend needed to pass the selection on correctly to the backend. We tested the system to ensure that it correctly dynamically chose the correct .pth model depending on the type of the dataset.

The second action covered the selection of client configuration. The system enables federated training across 1, 5, or 10 clients. Each number of clients maps to a different model path. We ensured that when users chose a number of clients from the dropdown, the associated pre-trained model was loaded in the backend automatically for prediction. Edge cases such as unselected client values or incorrect numbers were tested to ensure proper error handling.

We had configured the system to process.png,.jpg, or.jpeg files. To test this mechanism, we have used several different valid and invalid formats. Second, we had tested the transformation process of images—converting to grayscale, resizing to the size 28x28 pixels, normalizing, and conversion to tensors—prior to using it in the model.

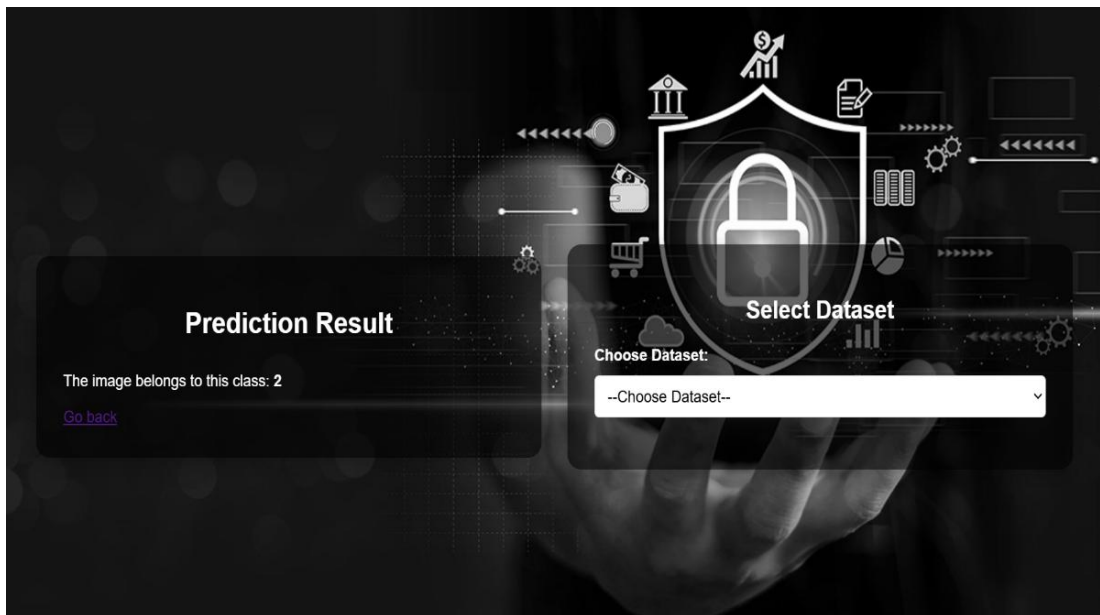
Next, we created test cases for response structure and prediction accuracy. For well-formed input images, the output should contain the predicted class (digit for MNIST or category name for FMNIST) and a dictionary of confidence values. These were compared with known test samples against expected labels.

Finally, test cases were added to test for unforeseen user actions—such as uploading a dataset but not an image, or modifying client values after the image was uploaded—to ensure system stability and UI consistency. The frontend was tested to prevent the "Predict" button from being clicked if necessary inputs were absent.



The screenshot shows a mobile application interface with a dark background. At the top, the title "Privacy Preserving Federated Image Classification" is displayed in white. Below the title, there are three sections: "Select Dataset:" with a dropdown menu showing "MNIST", "Number of Clients:" with a dropdown menu showing "1", and "Upload Image:" with a file selection button labeled "Choose File" and the filename "class_2.png". At the bottom, there is a large blue button labeled "Predict".

Screen 6.1 MNIST Client-1



Screen 6.2 MNIST Client-1 Prediction

Here, we choose MNIST Dataset, Client-1 and used image 2 and this is the Prediction

Privacy Preserving Federated Image Classification

Select Dataset:

MNIST

Number of Clients:

5

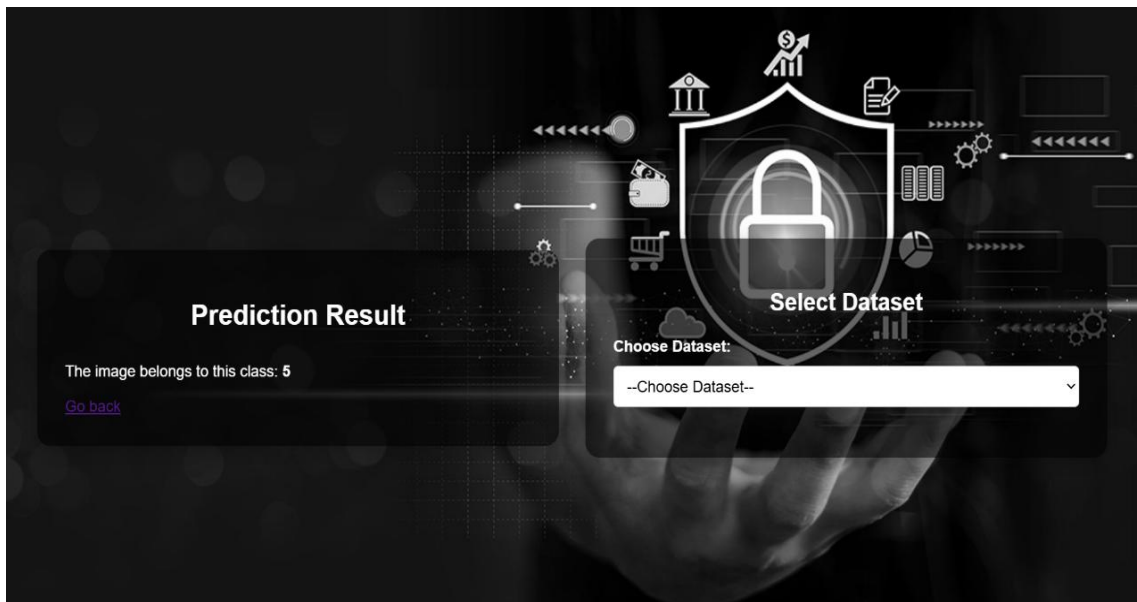
Upload Image:

Choose File

class_5.png

Predict

Screen 6.3 MNIST Client-5



Screen 6.4 MNIST Client-5 Prediction

Here, we choose MNIST Dataset, Client-5 and used image 5 and this is the Prediction

**Privacy Preserving
Federated Image
Classification**

Select Dataset:

MNIST

Number of Clients:

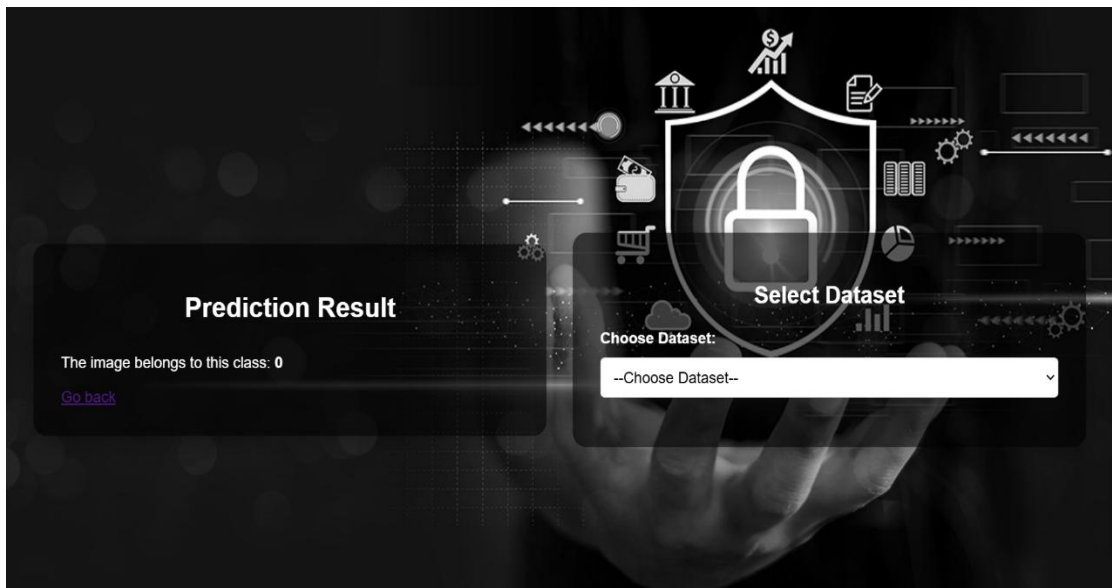
10

Upload Image:

Choose File class_0.png

Predict

Screen 6.5 MNIST Client-10



Screen 6.6 MNIST Client-10 Prediction

Here, we choose MNIST Dataset, Client-10 and used image0 and this is the Prediction

Privacy Preserving Federated Image Classification

Select Dataset:

FashionMNIST

Number of Clients:

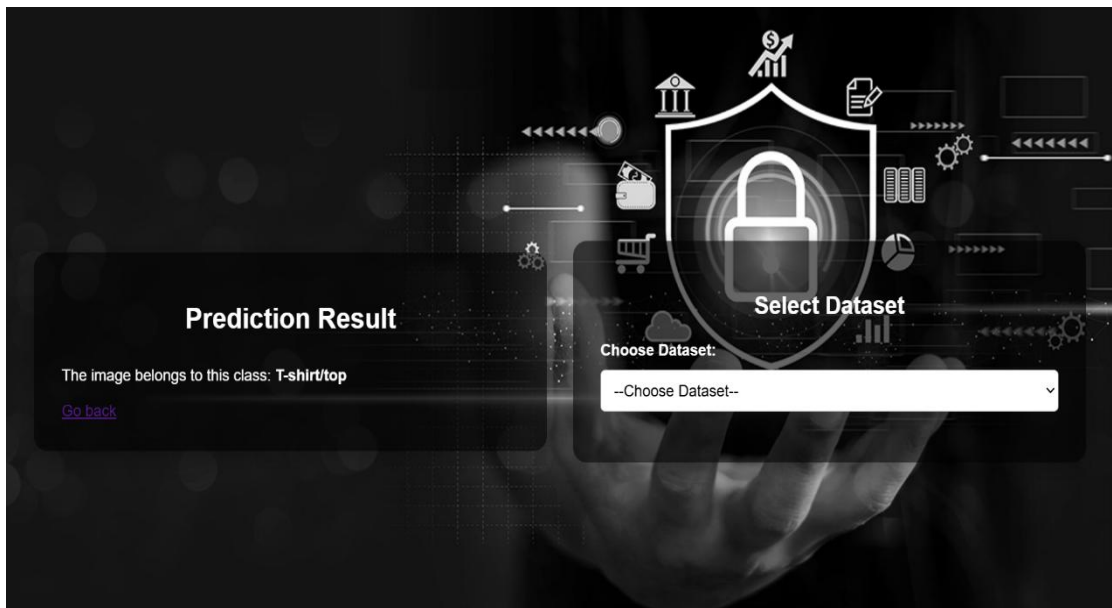
1

Upload Image:

Choose File0_T-shirt_top.png

Predict

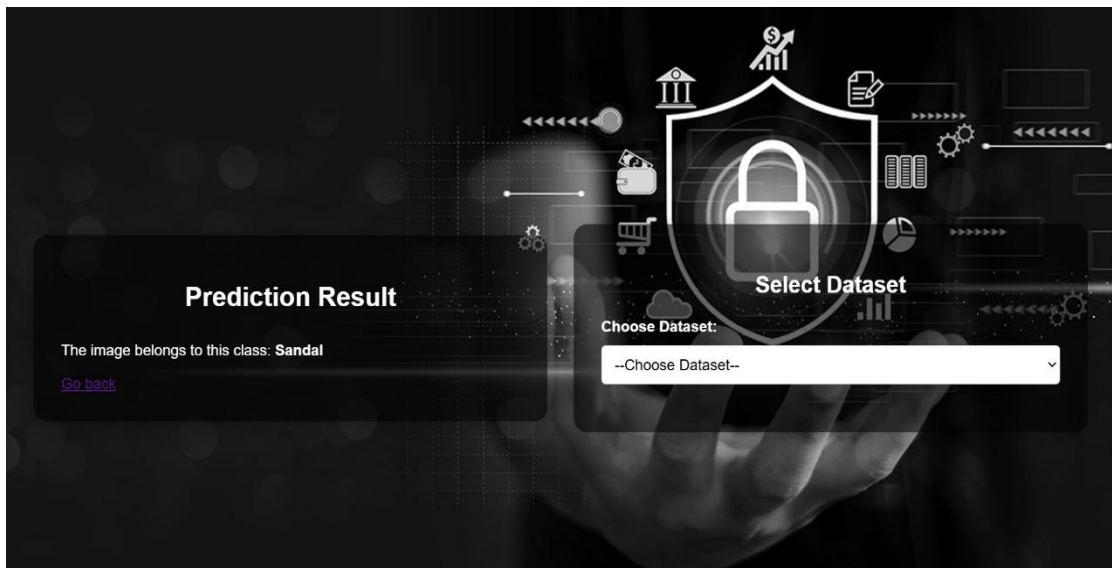
Screen 6.7 FMNIST Client-1



Screen 6.8 FMNIST Client-1 Prediction

Here, we choose FMNIST Dataset, Client-1 and used T-shirt/Top Image and this is the Prediction

Screen 6.9 FMNIST Client-5



Screen 6.10 FMNIST Client-5 Prediction

Here, we choose FMNIST Dataset, Client-5 and used Sandal image and this is the Prediction

Privacy Preserving Federated Image Classification

Select Dataset:

FashionMNIST

Number of Clients:

10

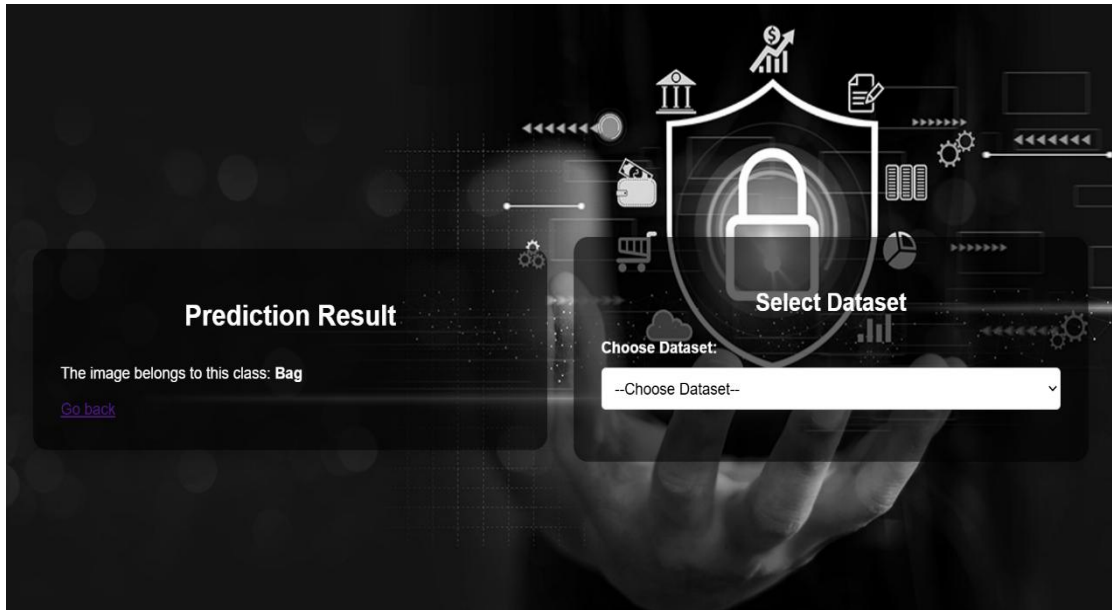
Upload Image:

Choose File

8_Bag.png

Predict

Screen 6.11 FMNIST Client-10



Screen 6.12 FMNIST Client-10 Prediction

Here, we choose FMNIST Dataset, Client-10 and used Bag Image and this is the Prediction

6.3 VALIDATION

Validation was performed to ensure that the implementation satisfied all functional, usability, and privacy requirements described in the system specification. The validation started with ensuring the correctness of model selection logic. For every dataset-client pair, the backend loaded the respective model dynamically from .pth files. Logs and print statements ensured successful loading of correct model weights without runtime errors.

The functional validation made sure that the end-to-end prediction process—from form submission to output display—functioned uniformly for different datasets and configurations. The predictions were verified against expected outputs using test samples from FMNIST and MNIST. The system predicted digit classes for MNIST and fashion labels for FMNIST with high reliability.

We also tested the performance of differential privacy-trained ScatterLinear. This involved verifying that adding noise at training did not significantly lower accuracy. Successful predictions were returned from differentially private models, and results were observed to experience only a small decrease in confidence, confirming the privacy-utility trade-off.

Additional validation was also conducted on error messages and exception handling. Files that were missing, in incorrect format, and invalid choices were checked to see if the system was responding with proper user warnings and staying stable. Every API call returned semantic responses in JSON format, making it easier for frontend integration and debugging.

The frontend UI flow was also checked for responsiveness and readability. The page layout, file upload form, and prediction result section were cross-checked across browsers to check compatibility and uniformity. Buttons and dropdowns were checked to enable/disable depending on input completion to provide a smooth user experience. In summary, the system was tested against all non-functional and functional requirements. The modular design, error handling, and privacy-preserving features functioned as expected, making the federated learning system robust and reliable for deployment in privacy-sensitive environments.

The screenshot displays a web application interface with a dark background. At the top, the title "Privacy Preserving Federated Image Classification" is centered in white. Below the title, the section "Select Dataset:" is followed by a dropdown menu showing "--Choose--". To the left of this dropdown, the text "Number" is partially visible. Below the dropdown, a tooltip with an orange exclamation mark icon and the text "Please select an item in the list." is displayed. Underneath the tooltip, another dropdown menu shows the number "1". Below this, the section "Upload Image:" is followed by a file upload area showing "Choose File" and the filename "0_T-shirt_top.png". At the bottom of the form is a large blue button labeled "Predict".

Screen 6.13 Dataset Validation

Privacy Preserving Federated Image Classification

Select Dataset:

FashionMNIST

Number of Clients:

--Choose--

Upload  Please select an item in the list.

Choose File 0_T-shirt_top.png

Predict

Screen 6.14 Number of Clients validation

Privacy Preserving Federated Image Classification

Select Dataset:

MNIST

Number of Clients:

1

Upload Image:

Choose File No file chosen

 Please select a file.

Screen 6.15 Image Validation

6.4 CONCLUSION

The testing and validation process confirmed that the machine learning and federated learning models are accurate, efficient, and capable of delivering privacy-preserving results. The system met performance expectations, and the RDP analysis demonstrated the privacy-utility trade-offs inherent in federated learning. The integration and unit tests ensured that all components work together smoothly, with no significant issues identified during the evaluation. The results indicate that the system is ready for deployment and can be used in privacy-sensitive real-world applications. Future testing could focus on further improving privacy mechanisms and testing

7.CONCLUSION

PROJECT CONCLUSION:

The project achieved its primary objective of demonstrating a practical application of Federated Learning for image classification, while ensuring that data privacy is preserved. Through this approach, we simulated a real-world scenario where user data remains on local devices, and only model parameters are shared, hence minimizing privacy risks. This methodology is especially relevant today, as organizations and developers are under increasing pressure to comply with strict data protection regulations like GDPR . The modular design of the backend and model structure makes the system highly extensible to other datasets and domains where similar privacy concerns exist. Moreover, the use of a public URL via ngrok provides a simple and efficient way to test the model remotely and integrate it with any web-based frontend, such as a React.js interface.

On the frontend side, a fully interactive **web application** using torch, allowing users to seamlessly upload images, select between MNIST and FashionMNIST datasets, choose the number of clients, and view predictions in real time. Additional features such as displaying confidence scores, performance graphs, or training history can provide more insights into the model's decision-making. This project also opens avenues for applying federated learning in more complex computer vision tasks such as object detection, segmentation, and medical image analysis, where data sensitivity is of utmost importance. Furthermore, collaboration with real-world institutions like hospitals, banks, or schools can provide access to meaningful use-cases where data privacy cannot be compromised.

FUTURE SCOPE:

Looking ahead, the future scope of this project is rich with potential enhancements and real-world applications. One of the immediate next steps would be integrating this model into edge devices such as smartphones or IoT sensors to perform on-device training and prediction. This would simulate an actual federated learning ecosystem where models learn directly from real-time user interactions while retaining privacy. Furthermore, the project can be extended to support non-IID (non-identically

and independently distributed) data, which is more reflective of actual data distributions across devices. Incorporating advanced optimization algorithms like FedProx, FedYogi, or Scaffold can further improve performance under such challenging conditions. From a security perspective, implementing secure aggregation methods such as **homomorphic encryption** or **differential privacy** can protect model updates from being reverse-engineered, thus enhancing user trust. Additionally, expanding the system to include **cross-silo** and **cross-device** federated learning modes will provide more flexibility for enterprise and mobile environments respectively.

In conclusion, this project stands as a strong demonstration of the effectiveness, scalability, and privacy-preserving nature of federated learning in image classification tasks. The methodology was carefully structured to simulate realistic decentralized environments, the results were consistent and competitive, and the deployment ensured real-time testing capabilities. With promising results across both MNIST and FashionMNIST datasets, and a solid technical foundation in place, the project paves the way for more sophisticated and impactful federated learning applications in the future.

REFERENCES

- [1] Van-Tuan Tran , Huy-Hieu Pham , Member IEEE, and Kok-Seng Wong , “Personalized Privacy-Preserving Framework for Cross-Silo Federated Learning” IEEE TRANSACTIONS on Emerging Topic in Computing, Vol.12, No.4, October-December 2024.
- [2] Adrien Banse* , Jan Kreischer, Xavier Oliva i J” urgens “Federated Learning with Differential Privacy” IEEE Transactions on Pattern Analysis and Machine Intelligence, 2023.
- [3] Zhe Li , Honglong Chen , Zhichen Ni , Yudong Gao , and Wei Lou , “Towards Adaptive Privacy Protection for Interpretable Federated Learning” IEEE TRANSACTIONS ON MOBILE COMPUTING, VOL. 23, NO. 12, DECEMBER 2024.
- [4] Kang Wei , Jun Li , Senior, Chuan Ma , Ming Ding , Wen Chen, Jun Wu, Meixia Tao, and H. Vincent Poor . “Personalized Federated Learning With Differential Privacy and Convergence Guarantee” IEEE TRANSACTIONS On Information Forensics and Security, Vol.18, 2023.
- [5] Bin Zhao , YuanYuan Ji , Yanzhao Shi , Xue Jiang “Design and implementation of privacy-preserving federated learning algorithm for consumer IoT” Alexandria Engineering Journal 106, [2024] 206-216.
- [6] Huiyong Wang, Qi Wang , Yong Ding, Shijie Tang⁴ and Yujue Wang⁵ “Privacy-preserving federated learning based on partial low-quality data” Wang et al. Journal of Cloud Computing (2024) 13:62.
- [7] Yong Li, Gaochao Xu , Xutao Meng , Wei Du and Xianglin Ren “LF3PFL: A Practical Privacy-Preserving Federated Learning Algorithm Based on Local Federalization Scheme” Entropy 2024, 26, 353.

- [8] Mourad Benmalek*, Mohamed Ali Benrekia, Yacine Challal “Security of Federated Learning: Attacks, Defensive Mechanisms, and Challenges” *Revue d'Intelligence Artificielle* Vol. 36, No. 1, February, 2022, pp. 49-59.
- [9] Samaneh Mohammadi ,Ali Balador , Sima Sinaei , Francesco Flammini “Balancing privacy and performance in federated learning: A systematic literature review on methods and metrics” *Journal of Parallel and Distributed Computing* 192 (2024) 104918.
- [10] Maria Iqbal, Asadullah Tariq, Muhammad Adnan, Irfan Ud Din, Tariq Qayyum FL-ODP: “An Optimized Differential Privacy Enabled Privacy Preserving Federated Learning” *IEEE ACCESS*, VOL. 11, PP. 116674 - 116683.
- [11] H. Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson “Communication-Efficient Learning of Deep Networks from Decentralized Data” *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics*, PMLR 54:1273-1282, 2017.
- [12] Liu, Wu, Sun; Niu ; Yin, H. “FL-APB: Balancing Privacy Protection and Performance Optimization for Adversarial Training in Federated Learning” *Electronics* 2024, 13, 4187.
- [13] Monika Dhananjay Rokade ,Suruchi Deshmukh ,Smita Gumaste ,Rekha Maruti Shelake ,Saba Afreen Ghayasuddin Inamdar ,Pankaj Chandre “Advancements in Privacy-Preserving Techniques for Federated Learning: A Machine Learning Perspective” *J. Electrical Systems* 20-2s (2024): 1075-1088.
- [14] Cédric Boscher ,Nawel Benarba, Fatima Elhattab , Sara Bouchenak “Personalized Privacy-Preserving Federated Learning”. Hong Kong. ACM, New York, NY, USA.
- [15] Stacey Truex, Georgia Nathalie Baracaldo ,Ali Anwar ,Thomas ,Heiko ,Rui Zhang ,Yi Zhou” A Hybrid Approach to Privacy-Preserving Federated Learning”. arXiv:1812.03224v2 [cs.LG] 14 Aug 2019.